



Especificación de Requerimientos – Colitas felices

Iteración final

Alumnos: Julieta Melina Flores Morán (321088642) - José Antonio Zarco Romero (321102502)- Luis Ángel Casique Abraham (321266536) - Yolanda Ximena Sánchez Cervantes (319263886)

Profesora: Karla Socorro García Alcántara

Versión del Documento: 5.0

Fecha: 08 de junio de 2026

1. Introducción

1.1 Propósito del documento

Este documento describe la evolución de los requerimientos funcionales y no funcionales del sistema Colitas Felices en el contexto de la Iteración final del proyecto. Su propósito es reflejar el estado actual del sistema implementado, funcionando como referencia técnica y funcional tanto para el equipo de desarrollo como para stakeholders.

Esta versión documenta el sistema completo que integra back-end e interfaz de usuario con funcionalidades extendidas más allá del manejo de usuarios y funcionalidades básicas de gestión y exploración de animales en adopción. La Iteración final implementa las funcionalidades que potencian el sistema como una opción integral de descubrimiento de mascotas en adopción cumpliendo todos los requerimientos previstos al inicio del desarrollo para el sistema.

Asimismo, este documento sirve como base de conocimiento para futuras iteraciones, permitiendo mantener coherencia entre el diseño del sistema y su implementación real.

1.2 Alcance del sistema

El sistema Colitas Felices es una aplicación web orientada a facilitar la adopción responsable de perros y gatos en el territorio mexicano, sirviendo como punto de contacto entre entidades cuidadoras de animales y personas interesadas en adoptar. Cuando un usuario hace clic en "Me interesa", el sistema envía automáticamente una notificación por correo al cuidador con

los datos de contacto del interesado, permitiendo que ambas partes continúen el proceso de adopción de manera autónoma.

En la Iteración final, el sistema evoluciona incorporando los requerimientos RF18–RF23. Las funcionalidades implementadas en esta iteración corresponden a:

- Autenticación de dos factores (2FA) mediante código enviado por correo electrónico
- Verificación de correo electrónico al registrarse
- Recuperación de contraseña mediante enlace por correo
- Bloqueo temporal de cuenta por intentos fallidos de inicio de sesión
- Panel de administración para gestión de reportes de publicaciones
- Registro de actividad (log de auditoría) de acciones relevantes en el sistema

Estas funcionalidades complementan el sistema de adopciones funcional desarrollado en iteraciones anteriores, fortaleciendo la seguridad de las cuentas, incorporando moderación de contenido y mejorando la trazabilidad de las acciones dentro de la plataforma.

1.3 Definiciones relevantes

1. **Usuario:** Persona registrada que accede al sistema con credenciales (rol: cuidador o interesado).
 - a. **Cuidador:** Usuario que publica y gestiona fichas de animales en adopción.
 - b. **Interesado:** Usuario que busca adoptar un animal.
2. **Usuario nuevo:** Persona que no cuenta con una cuenta registrada y desea crear un perfil dentro del sistema.
3. **Información personal:** Datos asociados a un usuario que permiten su identificación dentro del sistema, tales como nombre, correo electrónico y otros atributos definidos en el modelo de datos.
4. **Rol de usuario:** Atributo asociado a cada usuario que indica su tipo dentro del sistema (por ejemplo, cuidador o interesado).
5. **Autenticación:** Proceso mediante el cual el sistema verifica la identidad de un usuario a partir de sus credenciales.
6. **Usuario autenticado:** Usuario que ha iniciado sesión correctamente y cuenta con acceso a funcionalidades protegidas del sistema.
7. **Manifestación de interés:** Acción mediante la cual un adoptante expresa interés en un animal, generando una notificación automática por correo al cuidador.

8. **Autenticación de dos factores (2FA):** Mecanismo de seguridad que requiere un segundo paso de verificación (código numérico de 6 dígitos enviado por correo) además de las credenciales para completar el inicio de sesión.
9. **Verificación de correo:** Proceso mediante el cual el sistema confirma que el correo electrónico proporcionado durante el registro pertenece al usuario, a través de un enlace enviado a dicha dirección.
10. **Administrador:** Usuario con privilegios especiales para gestionar reportes de publicaciones, pudiendo resolver (eliminar publicación) o desestimar reportes.
11. **Reporte:** Señalamiento realizado por un usuario sobre una publicación que considera inapropiada, maliciosa o fraudulenta, que queda pendiente de revisión por un administrador.
12. **Log de actividad:** Registro cronológico de acciones relevantes realizadas dentro del sistema con fines de auditoría y trazabilidad.

2. Actores del sistema

Actor: Usuario registrado.

Tipo: Interno.

Descripción: Persona que crea una cuenta dentro del sistema y accede a la plataforma utilizando sus credenciales para consultar información sobre animales disponibles para adopción y utilizar las funcionalidades que le ofrece el sistema.

Responsabilidades dentro del sistema:

- Registrarse en la plataforma proporcionando la información requerida.
- Iniciar sesión mediante sus credenciales.
- Gestionar la información de su perfil personal.
- Cerrar sesión.

Actor: Usuario cuidador (publicador de animales).

Tipo: Interno.

Descripción: Usuario registrado responsable de publicar y administrar las fichas de animales que se encuentran bajo su cuidado y que están disponibles para adopción dentro de la plataforma.

Responsabilidades dentro del sistema:

- Registrar animales disponibles para adopción.
- Ingresar la información correspondiente de cada animal (nombre, descripción, fotografías, raza, edad, estado de vacunación, esterilización, padecimientos y género).
- Editar la información de los animales registrados.
- Eliminar publicaciones de animales cuando ya no estén disponibles.
- Marcar animales como adoptados.
- Consultar la lista de animales que ha registrado, incluyendo fecha de publicación y número de interesados.

Actor: Usuario interesado en adopción (adoptante).

Tipo: Interno.

Descripción: Usuario registrado que utiliza la plataforma para explorar animales disponibles y expresar interés en adoptar alguno de ellos.

Responsabilidades dentro del sistema:

- Explorar animales disponibles mediante la vista de lista.
- Consultar la ficha informativa de los animales disponibles.
- Manifestar su interés en adoptar un animal mediante el botón "Me interesa".
- Retirar su manifestación de interés en cualquier momento.
- Consultar los animales a los cuales ha manifestado interés (mis favoritos).

Actor: Sistema de correo electrónico.

Tipo: Externo.

Descripción: Servicio externo encargado de enviar notificaciones automáticas por correo electrónico cuando un usuario manifiesta o retira interés en un animal.

Responsabilidades dentro del sistema:

- Enviar notificación al cuidador cuando un adoptante manifiesta interés en uno de sus animales, incluyendo los datos de contacto del interesado.
- Enviar notificación al cuidador cuando un adoptante retira su interés.
- Incluir al adoptante en copia en ambas notificaciones.

Actor: API's externas de información de razas (Cat & Dog API)

Tipo: Externo.

Descripción: Servicios externos que proporcionan información adicional sobre razas de perros y gatos para enriquecer las biografías informativas de los animales registrados en la plataforma.

Responsabilidades dentro del sistema:

- Proporcionar información general sobre razas de perros y gatos.
- Ofrece datos complementarios como temperamento, características físicas y recomendaciones de cuidado.
- Permitir que el sistema muestre información adicional en la ficha informativa del animal.

Actor: Administrador del sistema.

Tipo: Interno.

Descripción: Usuario con rol ADMINISTRADOR que gestiona los reportes de publicaciones inapropiadas realizados por otros usuarios. Tiene acceso a un panel exclusivo donde puede revisar, resolver o desestimar reportes.

Responsabilidades dentro del sistema:

- Acceder al panel de administración.
- Consultar la lista de reportes pendientes agrupados por publicación.
- Resolver un reporte (eliminando la publicación y notificando al cuidador por correo con el motivo).
- Desestimar un reporte (conservando la publicación sin cambios).

3. Requerimientos Funcionales

Los siguientes requerimientos funcionales corresponden a la totalidad de funcionalidades implementadas en el sistema hasta la iteración final. El sistema cubre la gestión completa de usuarios con seguridad reforzada (2FA, verificación de correo, bloqueo por intentos fallidos, recuperación de contraseña), la gestión de animales en adopción por parte de cuidadores, la visualización del catálogo con filtros avanzados y mapa interactivo, la manifestación de interés con notificación automática por correo electrónico, integración con APIs externas de razas y un panel de administración para moderación de contenido.

Requerimiento		Descripción	Criterios de aceptación
Gestión de usuarios			
RF1	Registro de usuarios	El sistema deberá permitir a nuevos usuarios registrarse en la plataforma mediante el ingreso de su información personal, con el fin de crear una cuenta única.	El sistema deberá solicitar la información necesaria para el registro del usuario. El sistema deberá validar que la información obligatoria esté completa. El registro requiere: nombre, nombre de usuario, correo electrónico, código postal, CURP, rol (cuidador o adoptante) y contraseña. El sistema deberá validar que el correo electrónico y el nombre de usuario no estén previamente registrados. El sistema deberá registrar al usuario en la plataforma si la información es válida. Las contraseñas deberán cumplir los siguientes requisitos de complejidad: mínimo 8 caracteres, al menos una letra mayúscula, al menos una letra minúscula, al menos un número y al menos un carácter especial. Al

			registrarse exitosamente, el sistema envía un correo de verificación al usuario (ver RF19); la cuenta no podrá iniciar sesión hasta completar la verificación.
RF2	Inicio de sesión	El sistema deberá permitir a los usuarios registrados autenticarse para acceder a la plataforma.	El sistema deberá solicitar credenciales de acceso al usuario. El sistema deberá validar que las credenciales correspondan a un usuario registrado cuyo correo haya sido verificado. El sistema deberá rechazar el acceso cuando las credenciales sean inválidas, cuando el correo no esté verificado (error 403), o cuando la cuenta esté bloqueada por intentos fallidos (error 423). El ingreso se realizará con correo electrónico y contraseña. Tras validar las credenciales, el sistema enviará un código de verificación 2FA por correo y requerirá su ingreso para otorgar el token de sesión (ver RF18). El sistema incrementará un contador de intentos fallidos por cada login incorrecto; tras superar el límite, bloqueará la cuenta temporalmente (ver RF21).
RF3	Cierre de sesión	El sistema deberá permitir a los usuarios cerrar su sesión activa en la plataforma.	El sistema deberá finalizar la sesión del usuario autenticado. El sistema deberá impedir el acceso a funcionalidades protegidas después del cierre de sesión.
RF4	Obtener usuario autenticado	El sistema deberá permitir al usuario autenticado consultar su información personal.	El sistema deberá permitir el acceso únicamente a usuarios autenticados. El sistema deberá mostrar la información correspondiente al usuario autenticado. El sistema no deberá permitir el acceso a información de otros usuarios.
RF5	Actualización de datos de usuario autenticado	El sistema deberá permitir al usuario autenticado actualizar su información personal.	El sistema deberá permitir únicamente a usuarios autenticados realizar la actualización. El sistema deberá validar la información enviada y guardarla. Debe permitirles cambiar su información personal: nombre(s), apellido paterno, apellido materno, código postal, correo y foto de perfil. Si el usuario modifica su correo electrónico, el sistema enviará un enlace de verificación al nuevo correo. El correo actual permanece activo hasta que el usuario confirme el nuevo mediante el enlace. La sesión no se interrumpe durante este proceso.

Gestión de animales

RF6	Publicación de animales en adopción	El sistema deberá permitir a los usuarios con rol CUIDADOR crear un perfil de animal disponible para adopción.	Registrar publicaciones de perfil de animal en adopción con los siguientes datos mínimos: nombre del animal, descripción general, fotografía(s) (al menos una), tipo de animal (perro o gato), raza o raza similar, edad aproximada, vacunación, esterilización, padecimientos y género. El sistema deberá asociar esta información con el perfil del cuidador que lo publica y su código postal. Todo animal registrado es visible para que lo descubran los adoptantes.
RF7	Control de estado de publicación	El sistema deberá permitir a los cuidadores marcar los animales que publicaron como adoptados, retirándolos de la lista pública de adopción.	Solo el usuario que registró el animal puede cambiar su estado a "Adoptado" desde la gestión de sus publicaciones. Los animales marcados como adoptados dejan de aparecer en la lista de animales disponibles. Los animales adoptados permanecen visibles en la lista personal del cuidador ("Mis mascotas") con indicador visual de su estado, pudiendo filtrarse por estatus. El cuidador puede revertir el estado a DISPONIBLE.
RF8	Lista de animales registrados por usuario	El sistema deberá permitir a los cuidadores consultar y administrar los animales que han registrado.	Solo debe ser visible para el cuidador propietario. Debe permitirle ver los animales que registró. Debe permitir editar y eliminar los animales registrados. Puede visualizar fecha de publicación y número de interesados por cada animal.
Consulta de animales			
RF10	Lista de animales disponibles	El sistema deberá ofrecer una vista de catálogo para que los usuarios naveguen por la totalidad de los animales registrados con estatus DISPONIBLE.	El sistema muestra una lista con todos los animales registrados disponibles para adopción. El usuario puede seleccionar un animal de la lista para acceder a su ficha informativa completa. El sistema permite filtrar la lista y ordenar por diversos criterios de interés.
RF11	Ficha informativa	El sistema deberá mostrar una ficha informativa detallada para cada animal registrado en la plataforma.	Cada ficha debe mostrar de forma clara: nombre, descripción, fotografía(s) con galería navegable, tipo de animal, raza, código postal, edad, género, estado de vacunación, esterilización y padecimientos. Para adoptantes: debe incluir un botón visible denominado "Me interesa" para

			iniciar el proceso de contacto. Para el cuidador propietario: debe incluir controles de edición, eliminación y cambio de estado. Deberá mostrar información adicional sobre (API): ● Información general de la raza (o raza similar) ● Recomendaciones de cuidado ● Características relevantes (temperamento, nivel de actividad, etc.) proporcionadas por APIS externas
RF12	Manifestar interés en un animal	El sistema maneja de forma automática el envío de correo de un usuario adoptante interesado en adoptar hacia el cuidador que ha publicado un animal.	Al hacer clic en "Me interesa", el sistema deberá enviar automáticamente una notificación al correo del cuidador con los datos de contacto del adoptante, incluyendo al adoptante en copia. El usuario interesado deberá poder retirar su manifestación de interés en cualquier momento y esto también debe ser notificado al cuidador por correo. Si el envío de correo falla al manifestar interés, el sistema hace rollback y no registra el interés. Si el envío falla al retirar interés, el retiro se completa igualmente.
RF13	Apartado de mis favoritos	El sistema debe mostrar al adoptante los animales en los que ha manifestado interés para seguimiento rápido.	El sistema mantiene una lista de animales favoritos del adoptante. El usuario puede retirar su interés desde esta vista, eliminando el animal de la lista.
RF14	Notificación de ingreso	El sistema enviará automáticamente un correo al usuario cada vez que inicie sesión exitosamente, informándole sobre el nuevo acceso a su cuenta.	El correo se envía tras cada login exitoso. Si el envío falla, el login no se ve afectado. El correo incluye nombre del usuario y mensaje de alerta de actividad. El sistema registra el intento de envío en los logs.
RF15	Eliminación de cuenta	El sistema permitirá al usuario autenticado solicitar la eliminación lógica de su cuenta. La cuenta quedará inhabilitada y no podrá usarse para iniciar sesión.	Solo el usuario autenticado puede eliminar su propia cuenta. El sistema realiza un soft delete estableciendo fecha_eliminado. El token de sesión queda invalidado. El usuario no puede volver a iniciar sesión. Se envía correo de confirmación al correo del usuario. Si la cuenta ya fue eliminada, el sistema rechaza la solicitud.
RF16	Historial de animales adoptados	El sistema permitirá a un cuidador autenticado consultar la lista de todos los animales que ha publicado y que han sido marcados como adoptados.	Solo accesible para usuarios con rol CUIDADOR. Retorna la lista de animales del cuidador con estatus ADOPTADO. Si no hay animales adoptados, retorna lista vacía. Requiere

			autenticación válida.
Monitoreo			
RF17	Reportar publicaciones inapropiadas	El sistema permitirá a los usuarios autenticados reportar publicaciones que consideren maliciosas, fraudulentas o que incumplan las políticas de la plataforma, proporcionando un motivo.	Cualquier usuario autenticado puede reportar una publicación. El reporte requiere un motivo (texto obligatorio). El sistema registra el reporte con estado PENDIENTE. Un usuario no puede reportar el mismo animal más de una vez. El usuario puede retirar su reporte. Los reportes quedan en espera de revisión por un administrador (RF22).
Seguridad de autenticación			
RF18	Autenticación de dos factores (2FA)	El sistema requerirá un código de verificación de 6 dígitos enviado por correo electrónico como segundo paso de autenticación tras validar las credenciales.	Al ingresar credenciales válidas, el sistema envía un código de 6 dígitos al correo del usuario. El código expira en 10 minutos. El usuario debe ingresar el código correcto para obtener su token de sesión. Si el código es incorrecto, se rechaza con error 401. Si el código expiró, se rechaza con error 410 y el usuario debe iniciar el proceso nuevamente. El sistema no otorga token de sesión sin completar la verificación 2FA.
RF19	Verificación de correo electrónico	El sistema enviará un correo con enlace de verificación al momento del registro. La cuenta no podrá iniciar sesión hasta que el correo sea verificado.	Al registrarse, el sistema genera un token de verificación y envía un correo con enlace. El usuario debe hacer clic en el enlace para activar su cuenta. Si intenta hacer login sin verificar, el sistema rechaza con error 403 y mensaje "Debes verificar tu correo antes de iniciar sesión". El enlace de verificación activa la cuenta al ser accedido (endpoint GET). Las cuentas que no completen la verificación de correo dentro de las 24 horas posteriores al registro serán eliminadas automáticamente, liberando el correo, CURP y nombre de usuario para un nuevo registro.
RF20	Recuperación de contraseña	El sistema permitirá a un usuario solicitar el restablecimiento de su contraseña mediante un enlace enviado a su correo electrónico registrado.	El usuario ingresa su correo en la vista de recuperación. El sistema envía un enlace con token de recuperación (si el correo existe). El mensaje de respuesta no revela si el correo está registrado. El enlace dirige a una vista donde el usuario ingresa su nueva contraseña. La nueva contraseña debe cumplir los mismos requisitos de complejidad que en el registro. Una vez restablecida, el usuario puede iniciar sesión con la nueva contraseña.
RF21	Bloqueo por intentos fallidos	El sistema bloqueará temporalmente la cuenta de un usuario tras múltiples intentos fallidos consecutivos de inicio de sesión.	Tras superar el límite de intentos fallidos, la cuenta se bloquea por 15 minutos. Durante el bloqueo, el sistema rechaza intentos de login con error 423 y mensaje "Cuenta bloqueada temporalmente. Intenta en 15 minutos." El contador se reinicia tras un login exitoso. El bloqueo se levanta automáticamente al cumplirse el tiempo.
Administración			

RF22	Panel de administración de reportes	El sistema proporcionará a los usuarios con rol ADMINISTRADOR un panel para gestionar los reportes de publicaciones inapropiadas.	Solo accesible para usuarios con rol ADMINISTRADOR. Muestra la lista de reportes con estado PENDIENTE agrupados por publicación. El administrador puede resolver un reporte (elimina la publicación y envía correo al cuidador con el motivo). El administrador puede desestimar un reporte (cambia estado a DESESTIMADO sin afectar la publicación). Al resolver, todos los reportes del mismo animal quedan resueltos.
RF23	Registro de actividad	El sistema registrará un log de acciones relevantes con fines de auditoría y trazabilidad.	El sistema almacena: identificador del usuario, descripción de la acción y fecha/hora. No se almacena información personal identificable (PII) en el log, solo el ID interno. Se registran al menos: inicio de sesión, eliminación de cuenta, resolución de reportes. El log no es accesible desde el frontend (solo persistencia interna).

4. Casos de Uso

CU1 – Registro de usuario (RF1, RF19)

Actores:

- Usuario nuevo
- Sistema de correo electrónico

Descripción:

Este caso de uso describe el proceso mediante el cual un usuario crea una cuenta en la plataforma enviando su información al sistema, incluyendo la verificación de correo electrónico.

Precondiciones:

- El usuario no cuenta con una cuenta registrada.
- El usuario puede realizar solicitudes al sistema.

Flujo Principal:

1. El usuario accede a la vista de registro y envía una solicitud de registro con su información.

2. El sistema recibe la solicitud.
3. El sistema verifica que la información esté completa y cumpla con el formato esperado.
4. El sistema verifica que el correo, CURP y nombre de usuario no estén registrados.
5. El sistema valida que la contraseña cumpla los requisitos de complejidad (mínimo 8 caracteres, mayúscula, minúscula, número y carácter especial).
6. El sistema cifra la contraseña con BCrypt.
7. El sistema registra al usuario en la base de datos con estado `verificado = false`.
8. El sistema genera un token de verificación y envía un correo al usuario con un enlace para verificar su cuenta.
9. El sistema confirma el registro exitoso e indica al usuario que debe verificar su correo antes de iniciar sesión.

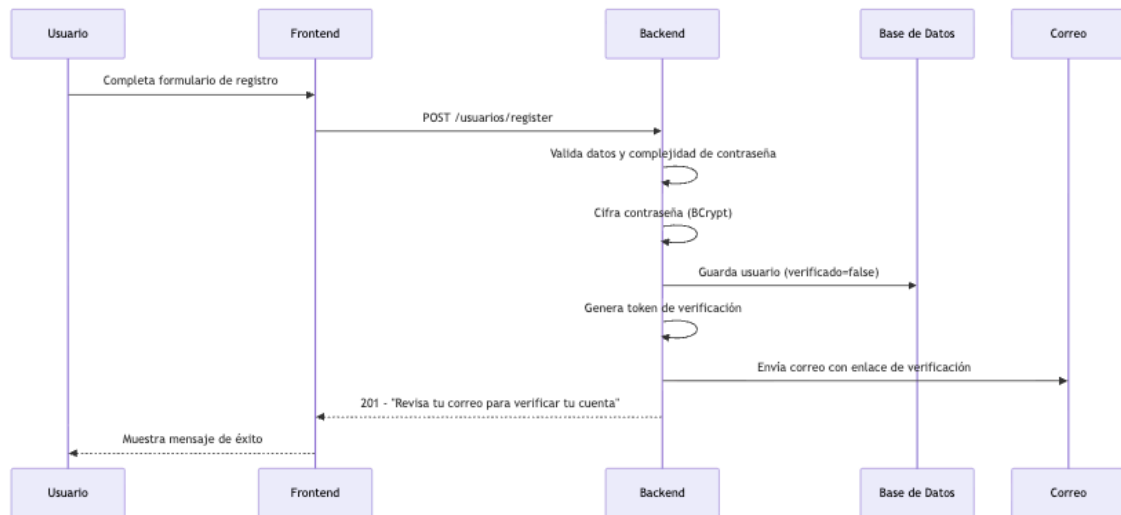
Flujos Alternativos:

- A1 – Información obligatoria faltante: El sistema muestra un mensaje de error indicando los campos pendientes. El sistema no registra al usuario.
- A2 – Correo electrónico, CURP o nombre de usuario ya registrado: El sistema muestra un mensaje indicando que ya están en uso. El sistema no registra al usuario.
- A3 – Formato de entrada inválido: El sistema muestra un mensaje de error. El sistema no registra al usuario.
- A4 – Contraseña no cumple requisitos: El sistema muestra un mensaje indicando: "La contraseña debe contener: mínimo 8 caracteres, al menos una mayúscula, al menos una minúscula, al menos un número, al menos un carácter especial." El sistema no registra al usuario.

Postcondiciones:

- La cuenta del usuario queda registrada con el rol seleccionado y `verificado = false`.
- El usuario recibe un correo con enlace de verificación.
- El usuario debe completar la verificación (CU21) antes de poder iniciar sesión.

Diagrama:



CU2 – Inicio de sesión (RF2, RF18, RF21)

Actores:

- Usuario registrado
- Sistema de correo electrónico

Descripción: Este caso de uso describe el proceso mediante el cual un usuario se autentica en el sistema con verificación de dos factores.

Precondiciones:

- El usuario cuenta con una cuenta registrada y activa.
- El correo del usuario ha sido verificado.
- La cuenta no está bloqueada.

Flujo Principal:

1. El usuario accede a la vista de login y envía sus credenciales (correo y contraseña).
2. El sistema recibe la solicitud.
3. El sistema verifica que la cuenta no esté bloqueada por intentos fallidos.

4. El sistema verifica que el correo esté verificado.
5. El sistema valida las credenciales (comparación BCrypt).
6. El sistema genera un código 2FA de 6 dígitos con expiración de 10 minutos.
7. El sistema envía el código al correo del usuario.
8. El frontend muestra la vista de ingreso de código 2FA.
9. El usuario ingresa el código recibido.
10. El sistema valida que el código sea correcto y no haya expirado.
11. El sistema genera un token UUID de sesión y lo almacena en la base de datos.
12. El sistema retorna el token al frontend.
13. El frontend almacena el token en sessionStorage y redirige al home.

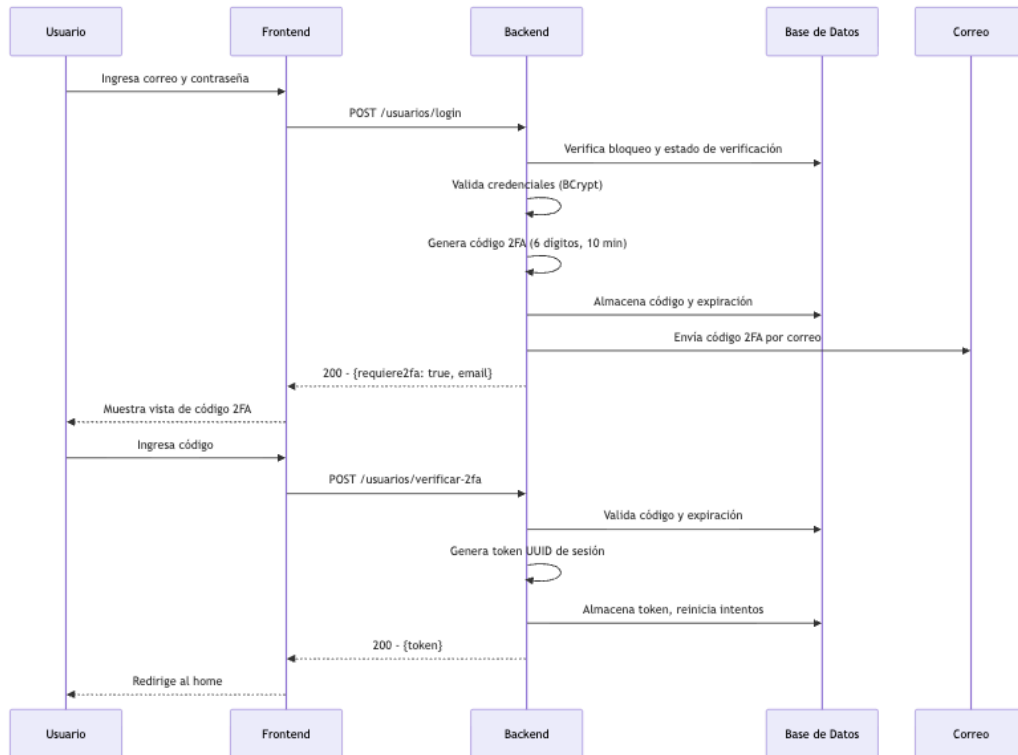
Flujos Alternativos:

- A1 – Credenciales incorrectas: El sistema incrementa el contador de intentos fallidos. Muestra mensaje de error genérico. Si se supera el límite, bloquea la cuenta 15 minutos.
- A2 – Correo no verificado: El sistema rechaza con error 403 "Debes verificar tu correo antes de iniciar sesión."
- A3 – Cuenta bloqueada: El sistema rechaza con error 423 "Cuenta bloqueada temporalmente. Intenta en 15 minutos."
- A4 – Cuenta eliminada: El sistema rechaza con error 401.
- A5 – Código 2FA incorrecto: El sistema devuelve error 401 "Código incorrecto." El usuario puede reintentar.
- A6 – Código 2FA expirado: El sistema devuelve error 410 "El código ha expirado. Inicia sesión de nuevo."

Postcondiciones:

- El usuario queda autenticado en el sistema tras completar ambos factores.
- El contador de intentos fallidos se reinicia a 0.

Diagrama:



CU3 – Obtener usuario autenticado (RF4)

Actores:

- Usuario autenticado

Descripción: Este caso de uso describe cómo un usuario autenticado solicita su información personal al sistema.

Precondiciones:

- El usuario está autenticado.

Flujo Principal:

1. El usuario accede a la vista de perfil y envía una solicitud para obtener su información.
2. El sistema valida la autenticación del usuario.
3. El sistema recupera la información del usuario.

4. El sistema devuelve la información solicitada y se la muestra al usuario en la vista de perfil.

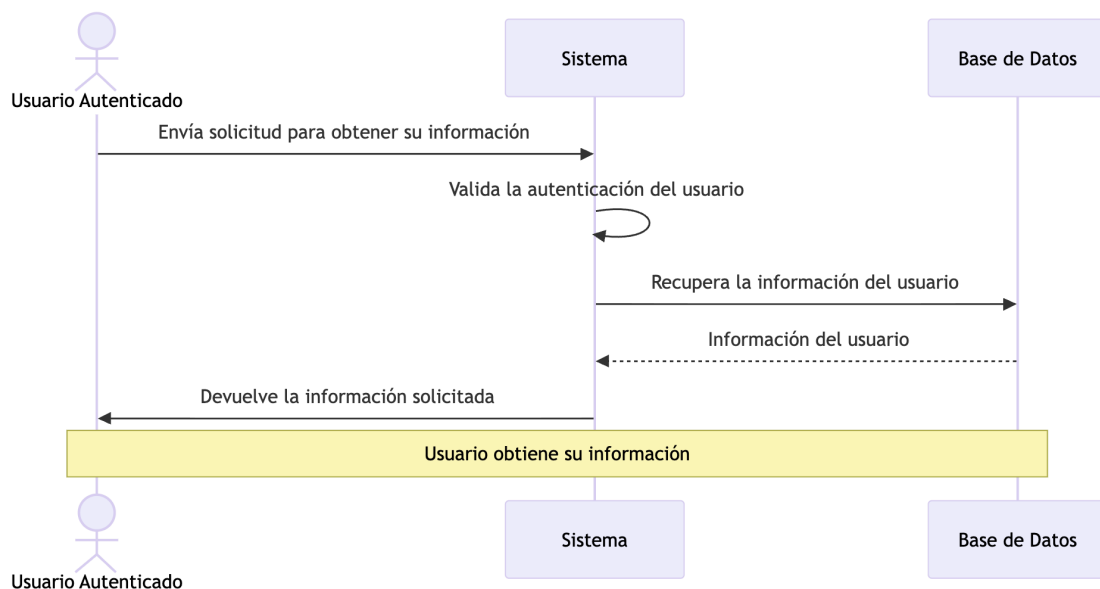
Flujos Alternativos:

- A1 - Usuario no autenticado: Se manda un mensaje de error explicando el problema y la acción de solución (autenticarse). El sistema rechaza la solicitud.

Postcondiciones:

- El usuario obtiene su información.

Diagrama:



CU4 – Actualización de usuario (RF5)

Actores:

- Usuario autenticado

Descripción: Este caso de uso describe el proceso mediante el cual un usuario modifica su información personal dentro del sistema.

Precondiciones:

- El usuario está autenticado.

Flujo Principal:

1. El usuario accede a la vista de perfil y envía una solicitud para editar su perfil.
2. El usuario edita su información personal.
3. El usuario envía la solicitud de actualización.
4. El sistema valida la autenticación del usuario.
5. El sistema valida los datos enviados.
6. El sistema actualiza la información en la base de datos.
7. El sistema confirma la actualización.

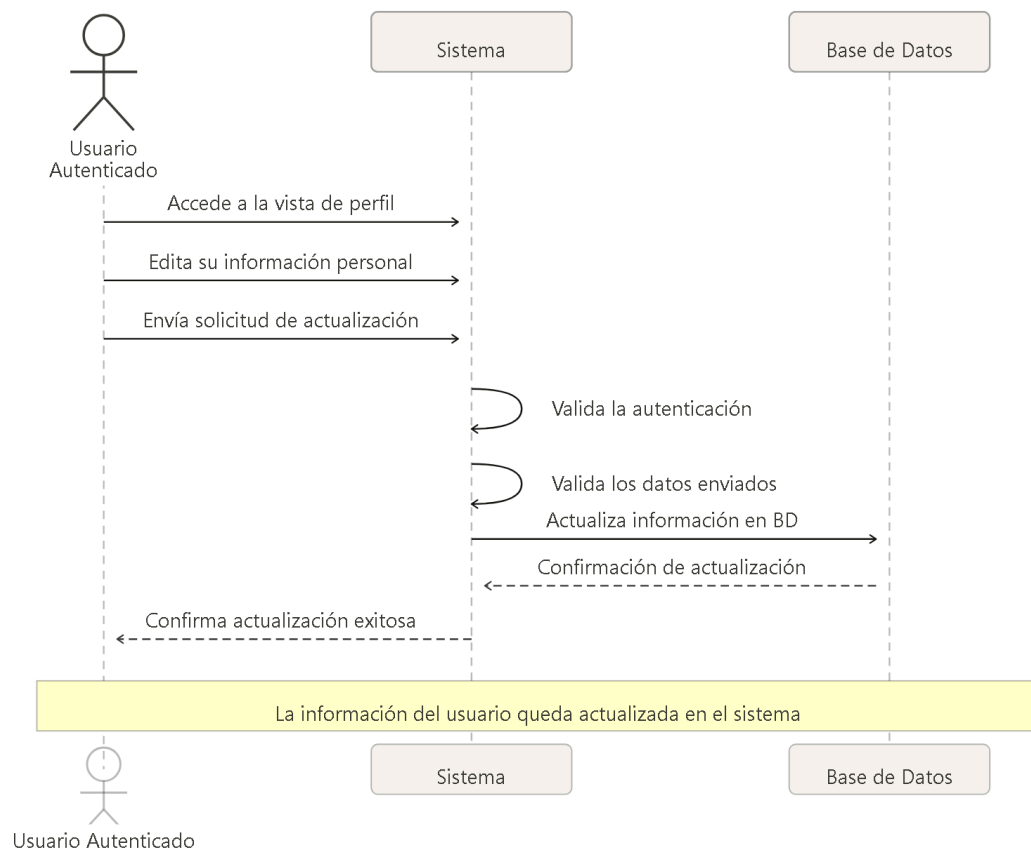
Flujos Alternativos:

- A1 - Usuario no autenticado: Se manda un mensaje de error explicando el problema y la acción de solución (autenticarse). El sistema rechaza la solicitud.
- A2 – Datos inválidos: El sistema muestra un mensaje de error indicando los campos incorrectos.
- A3 – Cambio de correo electrónico: Si el usuario modifica su email, el sistema guarda el nuevo correo como pendiente y envía un enlace de verificación a esa dirección. El correo anterior permanece activo y la sesión no se interrumpe. Al hacer clic en el enlace, el sistema aplica el cambio de correo.

Postcondiciones:

- La información del usuario queda actualizada en el sistema.

Diagrama:



CU5 – Cierre de sesión (RF3)

Actores:

- Usuario autenticado

Descripción: Este caso de uso describe el proceso mediante el cual un usuario finaliza su sesión en el sistema.

Precondiciones:

- El usuario está autenticado.

Flujo Principal:

1. El usuario selecciona la opción de cerrar sesión desde la barra de navegación y envía una solicitud para cerrar sesión.
2. El sistema recibe la solicitud.

3. El frontend finaliza la sesión del usuario e invalida el token de sessionStorage y redirige al usuario a la vista de login.

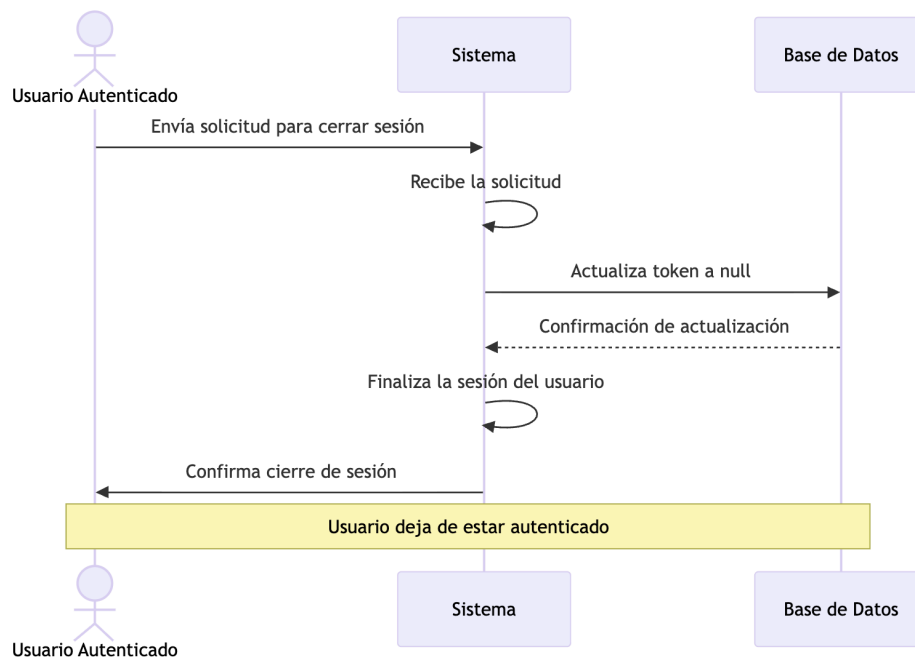
Flujos Alternativos:

- A1 - Usuario no autenticado: Se manda un mensaje de error explicando el problema y la acción de solución (autenticarse). El sistema rechaza la solicitud.

Postcondiciones:

- El usuario deja de estar autenticado.

Diagrama:



CU6 - Publicar animal (RF6)

Actores: Cuidador autenticado

Descripción: Este caso de uso describe el proceso mediante el cual un cuidador registra un nuevo animal en el catálogo del sistema para que esté disponible para adopción.

Precondiciones:

- El usuario está autenticado con rol CUIDADOR.

Flujo Principal:

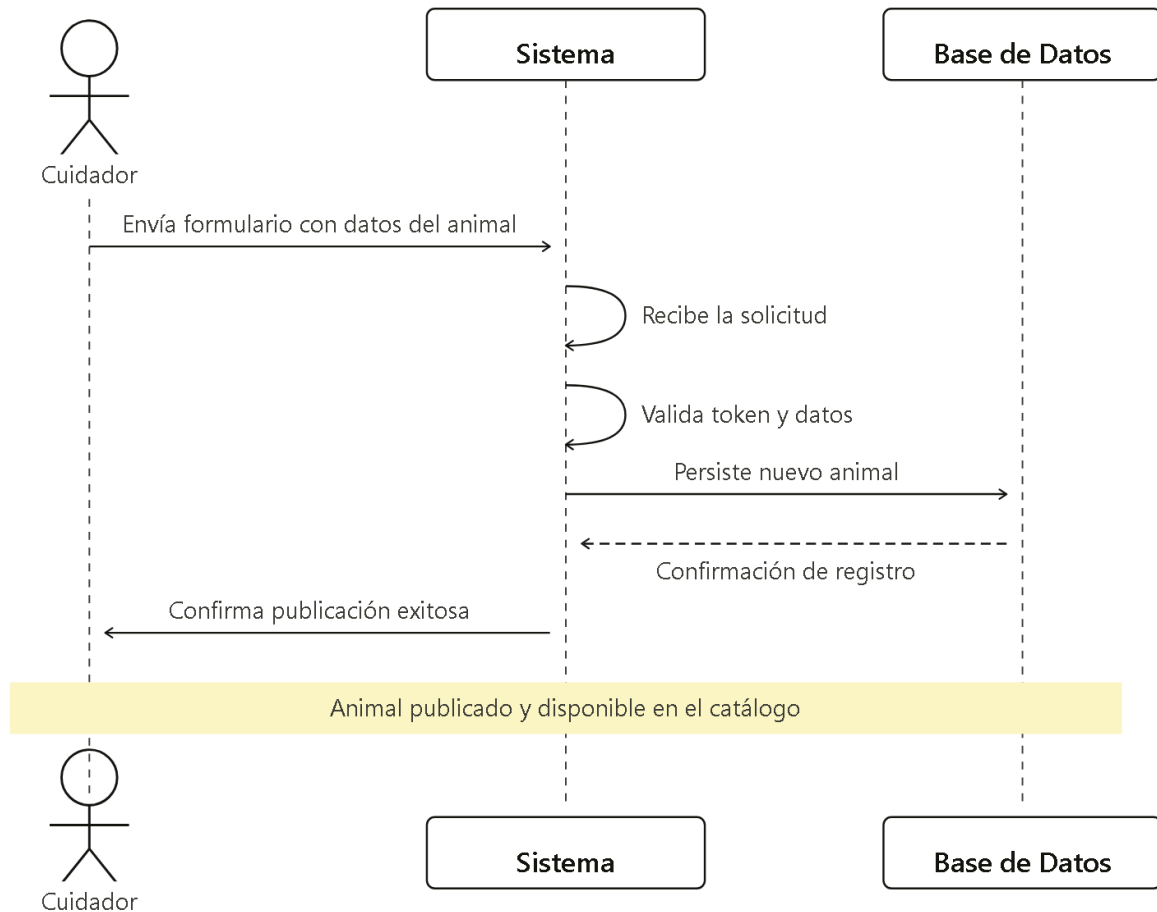
1. El cuidador accede a la vista de publicar animal y completar el formulario con los datos del animal.
2. El sistema recibe la solicitud.
3. El sistema valida el token de autenticación y los datos del formulario.
4. El sistema persiste el nuevo animal en la base de datos con estatus DISPONIBLE.
5. El sistema confirma la publicación exitosa y redirige al cuidador al catálogo.

Flujos Alternativos:

- A1 – Token ausente o inválido: El sistema devuelve un error 401 y rechaza la solicitud.
- A2 – Datos incompletos o inválidos: El sistema devuelve un error 400 indicando los campos incorrectos. El sistema no registra el animal.

Postcondiciones:

- El animal queda registrado en el sistema con estatus DISPONIBLE y es visible en el catálogo.



CU7 - Listar animales disponibles (RF10)

Actores:

- Usuario autenticado

Descripción: Este caso de uso describe el proceso mediante el cual un usuario accede al catálogo general para visualizar todos los animales que se encuentran disponibles para adopción.

Precondiciones:

- El usuario está autenticado.

Flujo Principal:

1. El usuario accede a la vista de catálogo de animales.

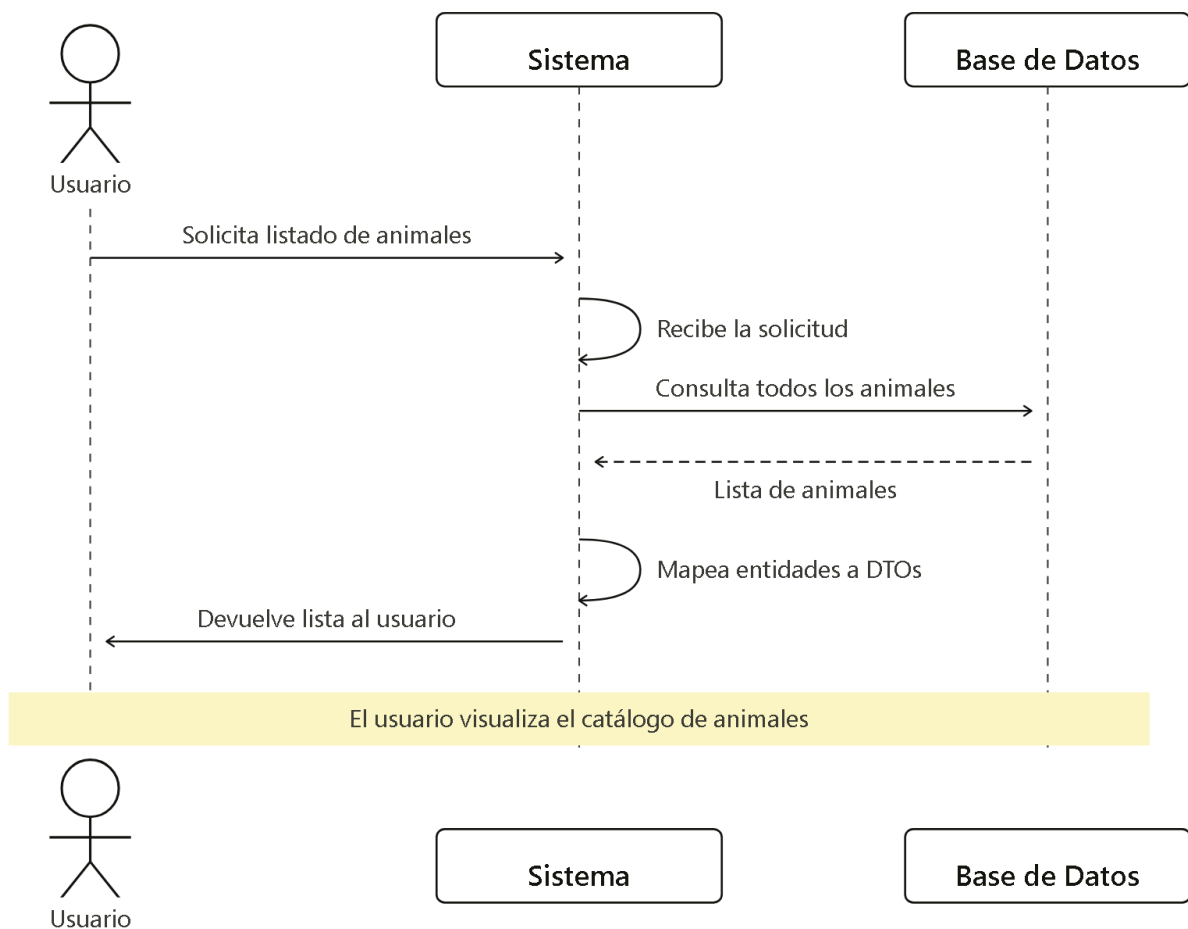
2. El sistema valida la autenticación del usuario.
3. El sistema recupera la lista de animales.
4. El sistema muestra la lista de animales disponibles junto con información resumida de cada publicación.
5. El usuario puede seleccionar un animal para consultar su ficha informativa completa.

Flujos Alternativos:

- A1 – Usuario no autenticado: El sistema muestra un mensaje indicando que debe iniciar sesión. El sistema rechaza la solicitud.
- A2 – No existen animales disponibles: El sistema muestra un mensaje indicando que no hay animales disponibles actualmente.

Postcondiciones:

- El usuario visualiza el catálogo actualizado de animales disponibles para adopción.



CU8 – Editar animal (RF6)

Actores:

- Cuidador autenticado

Descripción: Este caso de uso describe el proceso mediante el cual un cuidador modifica la información de un animal previamente registrado en la plataforma.

Precondiciones:

- El usuario está autenticado con rol CUIDADOR.
- El animal pertenece al usuario autenticado.

Flujo Principal:

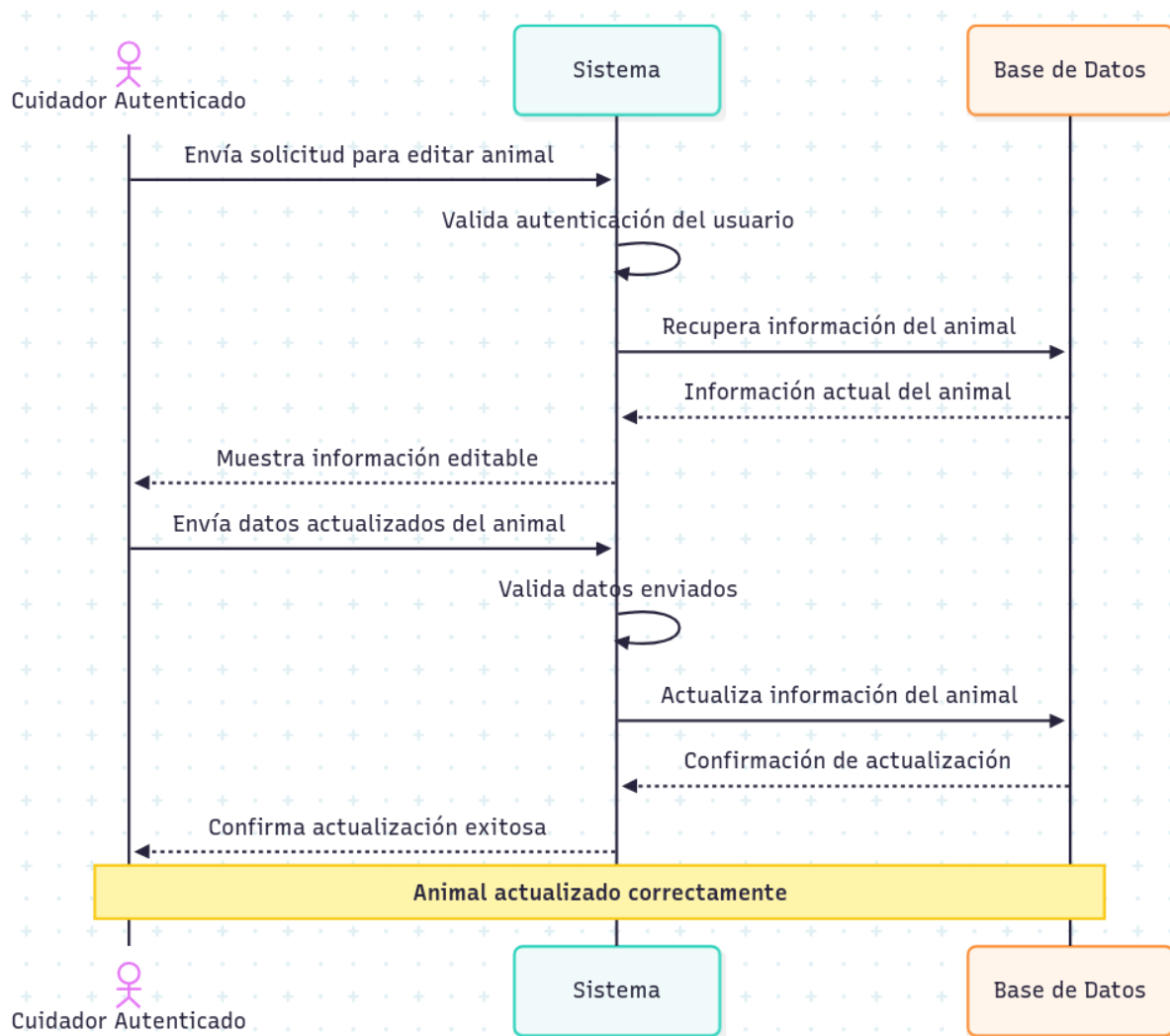
1. El cuidador accede a la vista de administración de sus publicaciones.
2. El cuidador selecciona un animal registrado y solicita editar su información.
3. El sistema recupera la información actual del animal.
4. El cuidador modifica los datos deseados.
5. El sistema valida la autenticación y los datos enviados.
6. El sistema actualiza la información del animal en la base de datos.
7. El sistema confirma la actualización exitosa.

Flujos Alternativos:

- A1 – Usuario no autenticado: El sistema devuelve un error de autenticación y rechaza la solicitud.
- A2 – Animal no pertenece al usuario: El sistema devuelve un error de autorización y no permite la modificación.
- A3 – Datos inválidos o incompletos: El sistema muestra un mensaje indicando los campos incorrectos. El sistema no actualiza la información.

Postcondiciones:

- La información del animal queda actualizada en el sistema.



CU9 – Eliminar animal (RF6)

Actores:

- Cuidador autenticado

Descripción: Este caso de uso describe el proceso mediante el cual un cuidador elimina una publicación de animal registrada en la plataforma.

Precondiciones:

- El usuario está autenticado con rol CUIDADOR.
- El animal pertenece al usuario autenticado.

Flujo Principal:

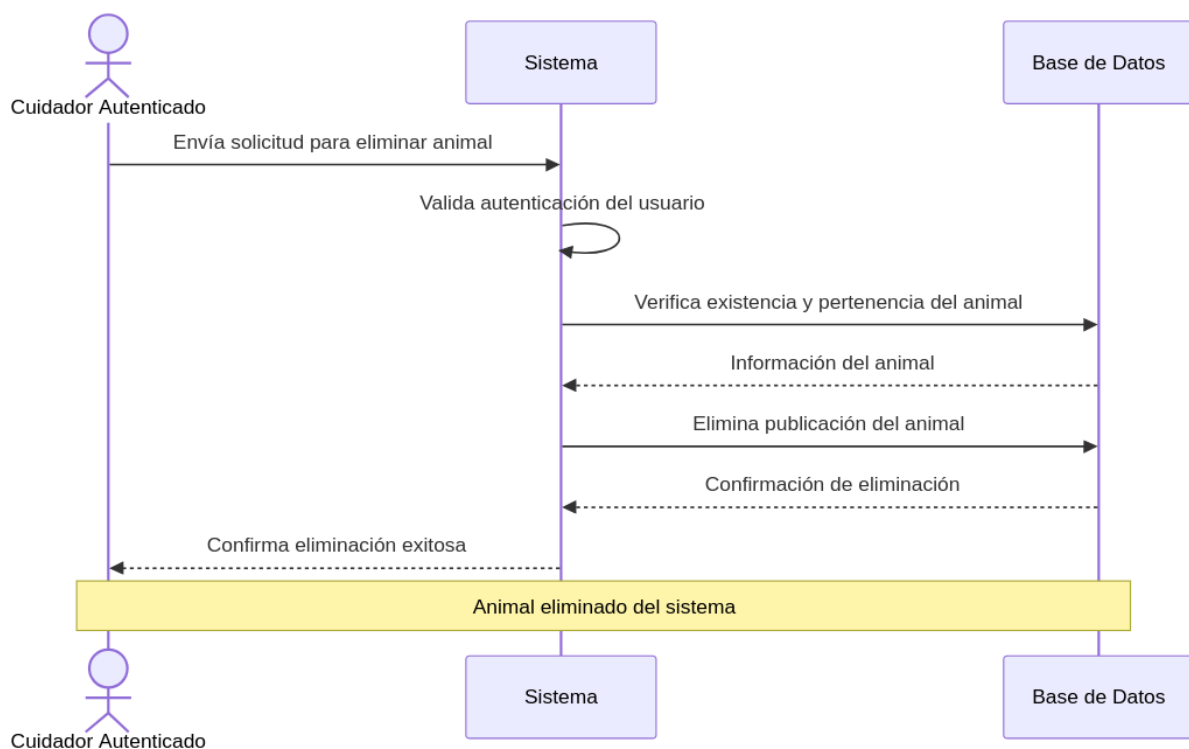
1. El cuidador accede a la vista de administración de publicaciones.
2. El cuidador selecciona un animal registrado y solicita eliminar la publicación.
3. El sistema valida la autenticación y la pertenencia del animal.
4. El sistema elimina la publicación del animal de la base de datos.
5. El sistema confirma la eliminación exitosa.

Flujos Alternativos:

- A1 – Usuario no autenticado: El sistema devuelve un error de autenticación y rechaza la solicitud.
- A2 – Animal no pertenece al usuario: El sistema devuelve un error de autorización y no permite la eliminación.
- A3 – Animal inexistente: El sistema muestra un mensaje indicando que la publicación no existe o ya fue eliminada.

Postcondiciones:

- La publicación del animal deja de existir en el sistema y deja de ser visible en el catálogo.



CU10 – Manifestar interés en un animal (RF12)

Actores:

- Adoptante autenticado
- Sistema de correo electrónico

Descripción: Este caso de uso describe el proceso mediante el cual un adoptante expresa su interés en adoptar un animal, desencadenando una notificación automática al cuidador responsable.

Precondiciones:

- El usuario está autenticado con rol ADOPTANTE.
- El animal existe en el sistema con estatus DISPONIBLE.
- No existe un registro de interés activo del usuario en ese animal al momento de la solicitud.

Flujo Principal:

1. El usuario visualiza la ficha de un animal y hace clic en el botón "Me interesa".
2. El sistema recibe la solicitud con el token de autenticación y el identificador del animal.
3. El sistema valida el token y verifica que el usuario sea ADOPTANTE.
4. El sistema verifica que el animal exista y esté disponible.
5. El sistema verifica que el usuario no haya manifestado interés previamente.
6. El sistema registra el interés en la base de datos.
7. El sistema envía automáticamente un correo al cuidador del animal con los datos de contacto del adoptante, incluyendo al adoptante en copia.
8. El sistema confirma el registro del interés al frontend.
9. El frontend muestra un modal informando al usuario que el cuidador será notificado.

Flujos Alternativos:

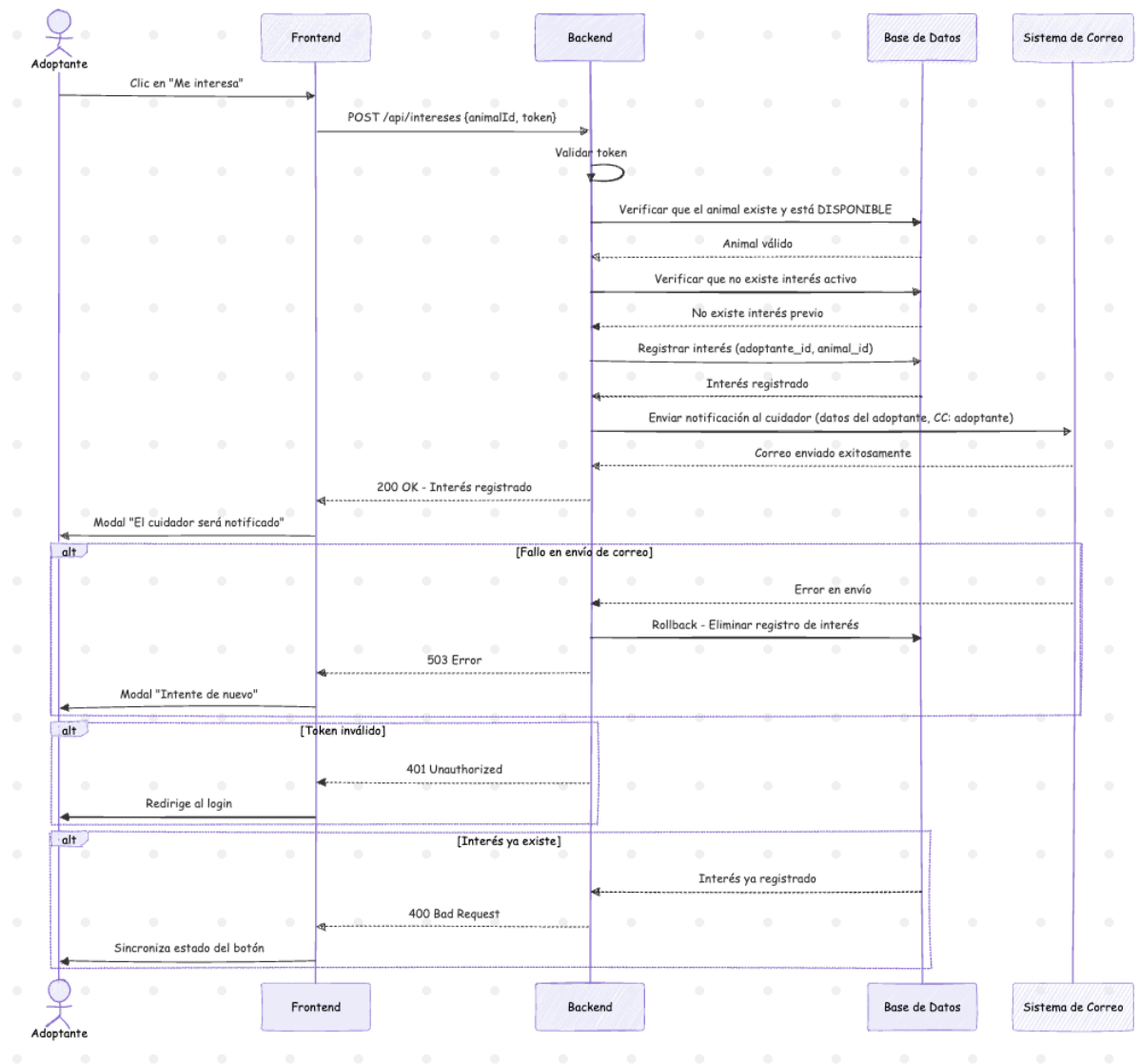
- A1 – Token ausente o inválido: El sistema devuelve un error 401 y rechaza la solicitud.

- A2 – Usuario con rol CUIDADOR: El sistema rechaza la solicitud con un error 400. El botón no se muestra en la interfaz para cuidadores.
- A3 – Animal no encontrado o ya adoptado: El sistema devuelve un error 400. El frontend actualiza el estado del botón indicando que el animal no está disponible.
- A4 – Interés ya registrado: El sistema devuelve un error 400 indicando que el interés ya existe. El frontend sincroniza el estado del botón.
- A5 – Fallo en el envío del correo: El sistema hace rollback del registro de interés y devuelve un error 503. El frontend informa al usuario que debe intentarlo de nuevo.

Postcondiciones:

- El interés queda registrado en la base de datos.
- El cuidador recibe una notificación por correo con los datos de contacto del adoptante.
- El animal aparece en la lista de favoritos del adoptante.

Diagrama:



CU11 – Retirar interés en un animal (RF12)

Actores: Adoptante autenticado, Sistema de correo electrónico

Descripción: Este caso de uso describe el proceso mediante el cual un adoptante retira su manifestación de interés en un animal, notificando automáticamente al cuidador.

Precondiciones:

- El usuario está autenticado con rol ADOPTANTE.
- El usuario tiene un interés registrado en el animal.

Flujo Principal:

1. El usuario hace clic en el botón "En favoritos" (estado activo) en la ficha del animal o en su lista de favoritos.
2. El sistema recibe la solicitud con el token de autenticación y el identificador del animal.
3. El sistema valida el token y verifica que exista un interés registrado.
4. El sistema elimina el registro de interés de la base de datos.
5. El sistema envía automáticamente un correo al cuidador informando que el adoptante retiró su interés, incluyendo al adoptante en copia.
6. El sistema confirma la eliminación al frontend.
7. El frontend muestra un modal informando al usuario que el cuidador ha sido notificado.

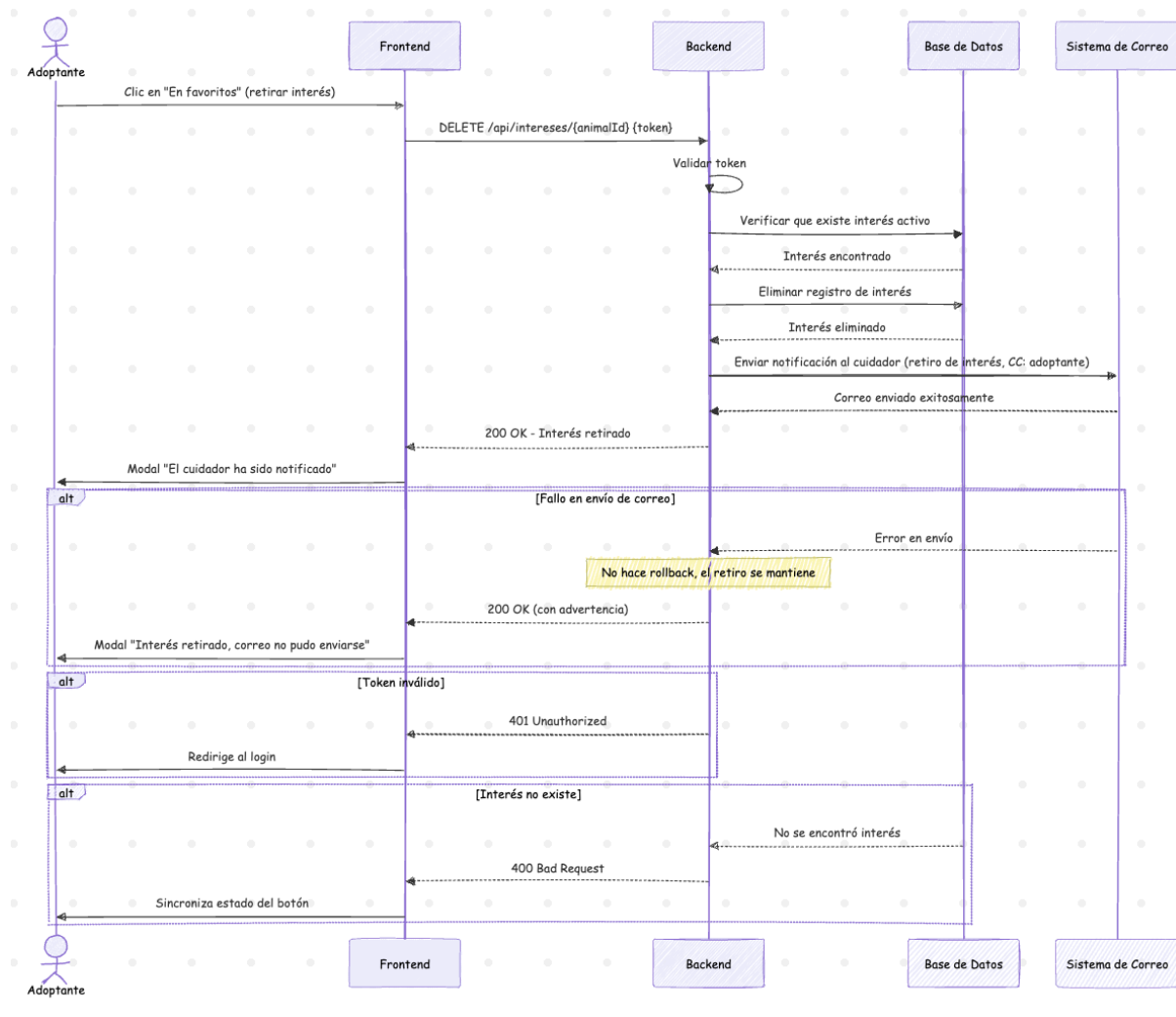
Flujos Alternativos:

- A1 – Token ausente o inválido: El sistema devuelve un error 401 y rechaza la solicitud.
- A2 – Interés no registrado: El sistema devuelve un error 400. El frontend sincroniza el estado del botón.
- A3 – Fallo en el envío del correo: El sistema registra el error, pero no hace rollback. El retiro de interés se completa y se notifica al usuario que el correo no pudo enviarse.

Postcondiciones:

- El interés queda eliminado de la base de datos.
- El cuidador recibe una notificación por correo informando del retiro.
- El animal deja de aparecer en la lista de favoritos del adoptante.

Diagrama:



CU12 – Consultar mis favoritos (RF13)

Actores: Adoptante autenticado

Descripción: Este caso de uso describe el proceso mediante el cual un adoptante consulta la lista de animales en los que ha manifestado interés.

Precondiciones:

- El usuario está autenticado con rol ADOPTANTE.

Flujo Principal:

1. El usuario accede a la vista "Mis favoritos" desde la barra de navegación.
2. El sistema recibe la solicitud con el token de autenticación.
3. El sistema valida el token.

- El sistema recupera todos los animales en los que el usuario ha manifestado interés, incluyendo información básica y foto de portada.
- El frontend muestra las tarjetas de los animales con el botón para retirar el interés.

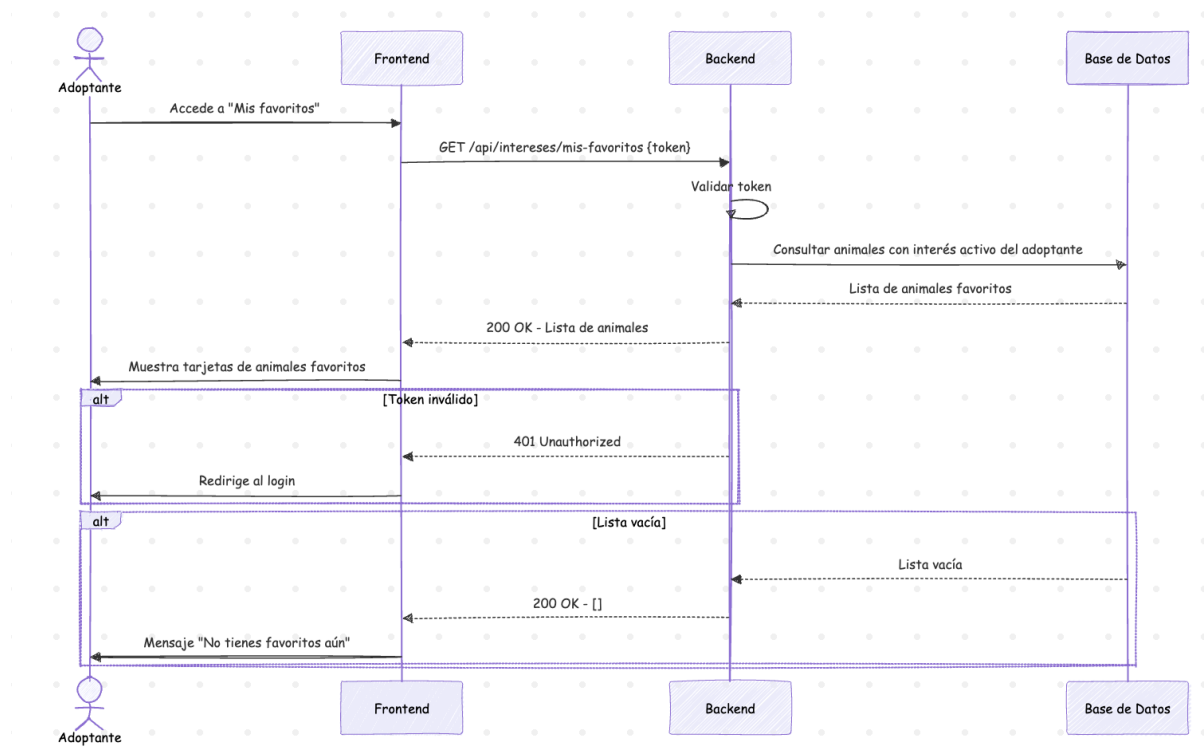
Flujos Alternativos:

- A1 – Token ausente o inválido: El sistema devuelve un error 401. El frontend redirige al login.
- A2 – Lista vacía: El sistema devuelve una lista vacía. El frontend muestra un mensaje indicando que no hay favoritos.

Postcondiciones:

- El usuario visualiza su lista actualizada de animales favoritos.

Diagrama:



CU13 – Marcar animal como adoptado (RF7)

Actores: Cuidador autenticado

Descripción: Este caso de uso describe el proceso mediante el cual un cuidador cambia el estatus de un animal a ADOPTADO, retirándolo del catálogo público.

Precondiciones:

- El usuario está autenticado con rol CUIDADOR.
- El animal pertenece al usuario autenticado.
- El animal tiene estatus DISPONIBLE.

Flujo Principal:

1. El cuidador accede a la vista "Mis mascotas" o al modal de detalle de un animal.
2. El cuidador selecciona la opción "Marcar como adoptado".
3. El sistema recibe la solicitud con el token de autenticación y el identificador del animal.
4. El sistema valida el token y verifica que el animal pertenezca al cuidador.
5. El sistema actualiza el estatus del animal a ADOPTADO en la base de datos.
6. El sistema confirma la actualización al frontend.
7. El frontend actualiza visualmente la tarjeta del animal con indicador de adoptado y lo retira del catálogo público.

Flujos Alternativos:

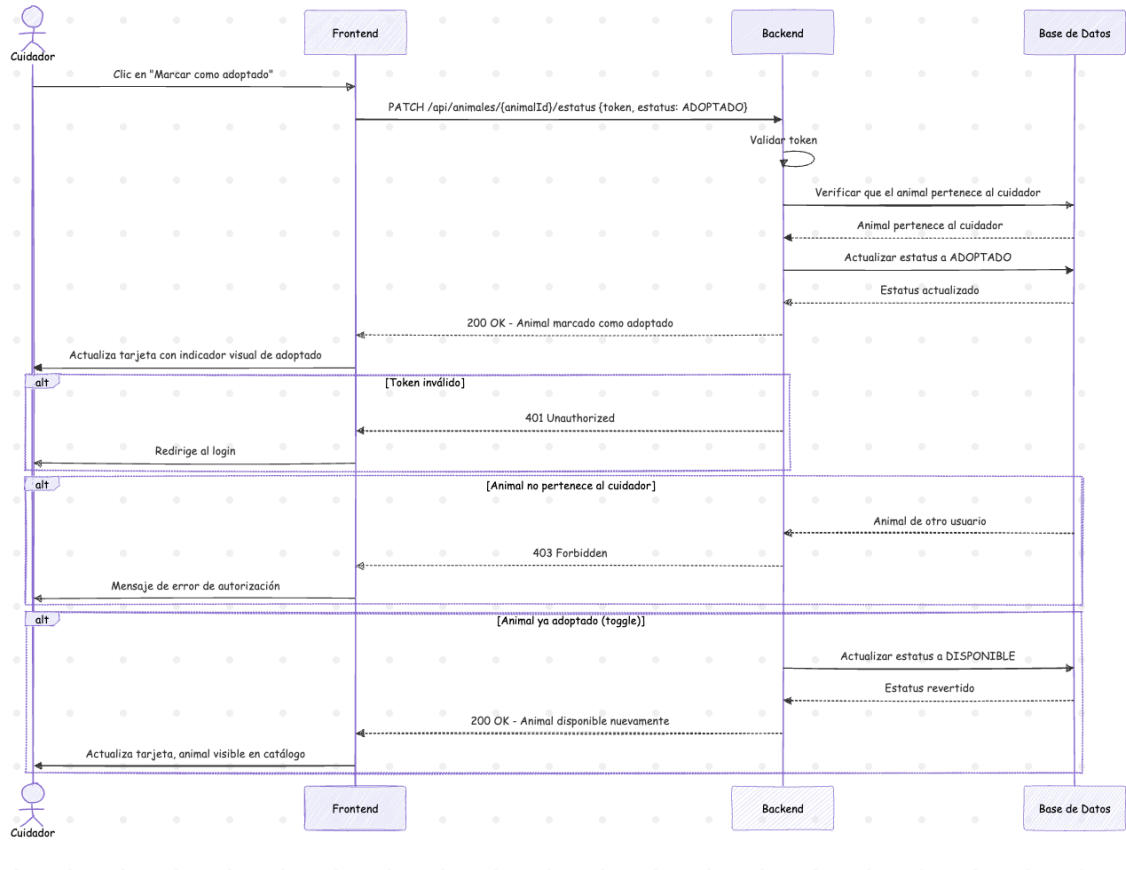
- A1 – Token ausente o inválido: El sistema devuelve un error 401 y rechaza la solicitud.
- A2 – Animal no pertenece al usuario: El sistema devuelve un error 403 y no permite la modificación.
- A3 – Animal ya adoptado: El sistema procesa la solicitud como "Desmarcar adoptado", revirtiendo el estatus a DISPONIBLE y haciéndolo visible nuevamente en el catálogo.

Postcondiciones:

- El animal queda con estatus ADOPTADO y deja de aparecer en el catálogo público.

- El animal permanece visible en "Mis mascotas" del cuidador con indicador visual de adoptado.

Diagrama:



CU14 – Consultar ficha informativa de un animal (RF11)

Actores: Usuario autenticado

Descripción: Este caso de uso describe el proceso mediante el cual un usuario accede a la información completa de un animal registrado en la plataforma.

Precondiciones:

- El usuario está autenticado.
- El animal existe en el sistema.

Flujo Principal:

1. El usuario selecciona un animal desde el catálogo, la vista de exploración o su lista de favoritos.
2. El sistema recibe la solicitud con el identificador del animal.
3. El sistema recupera la información completa del animal: nombre, descripción, fotografías, especie, raza, edad, género, estatus, esterilización, vacunas, padecimientos y fecha de publicación.
 - 3.b. El sistema muestra también información adicional sobre la raza del animal (temperamento, origen, esperanza de vida y características), obtenida de fuentes externas y traducida al español.
4. Si el usuario es el cuidador propietario, el sistema incluye además el número de adoptantes interesados.
5. El frontend muestra la ficha en un modal con galería de fotos navegable, sección de vacunas y padecimientos, y el botón "Me interesa" (para adoptantes) o los controles de gestión (para el cuidador propietario).

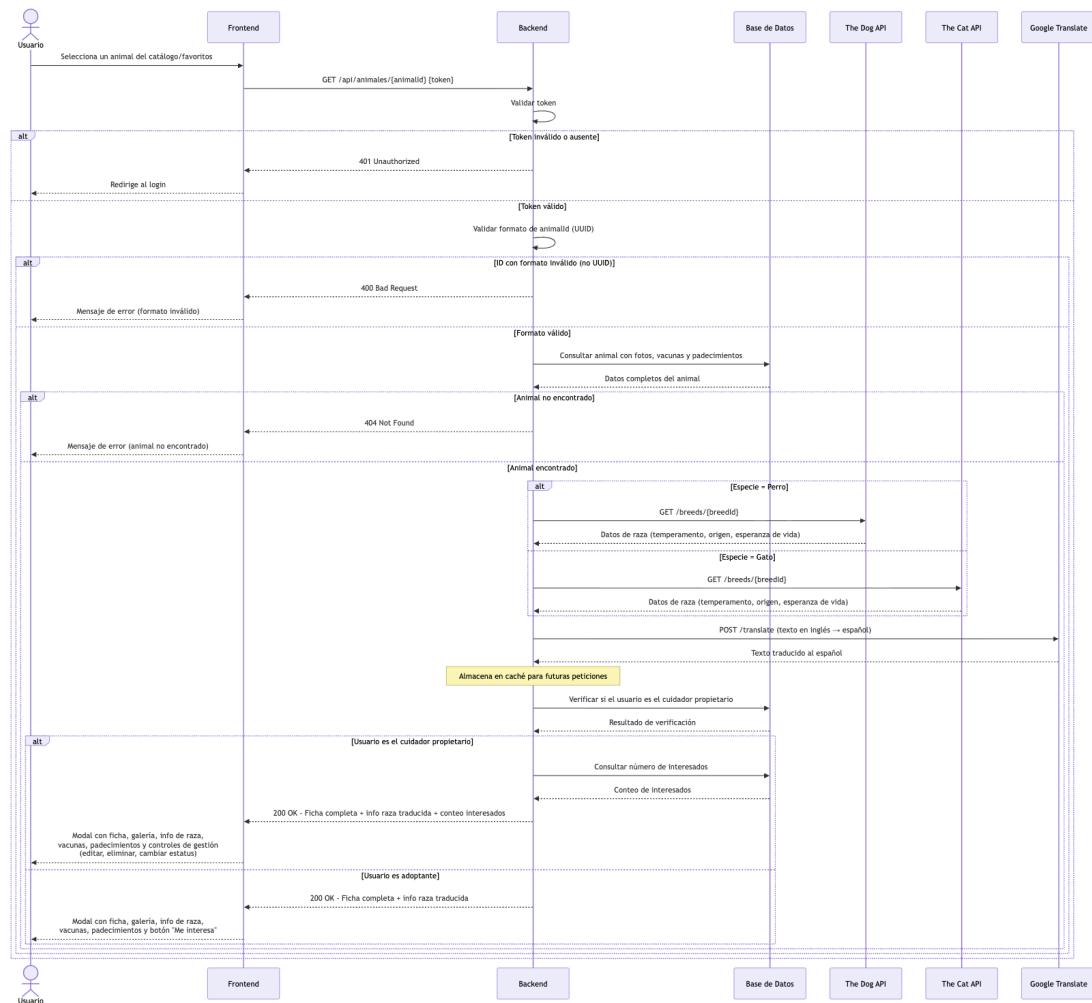
Flujos Alternativos:

- A1 – Token ausente o inválido: El sistema devuelve un error 401. El frontend redirige al login.
- A2 – Animal no encontrado: El sistema devuelve un error 404. El frontend muestra un mensaje de error.
- A3 – ID de animal con formato inválido (no UUID): El sistema devuelve un error 400.

Postcondiciones:

- El usuario visualiza la información completa del animal.
- Si es adoptante, puede manifestar o retirar su interés desde la misma ficha.
- Si es el cuidador propietario, puede editar, eliminar o cambiar el estatus del animal desde la misma ficha
- La información de la raza consultada queda almacenada en memoria del servidor para peticiones futuras, reduciendo la latencia y la dependencia de APIs externas.

Diagrama:



CU15 – Filtrar animales de interés (RF10)

Actores: Usuario autenticado con rol ADOPTANTE

Descripción: Este caso de uso describe el proceso mediante el cual un usuario con rol ADOPTANTE aplica filtros sobre el catálogo de animales en adopción para encontrar mascotas que se ajusten a sus preferencias y condiciones.

Precondiciones:

- El usuario está autenticado con rol ADOPTANTE.
- Existen animales disponibles registrados en el sistema.

Flujo Principal:

1. El usuario accede a la vista de exploración.
2. El sistema muestra el catálogo de animales disponibles con los filtros por defecto (sin restricciones).
3. El usuario selecciona uno o más criterios de filtrado: especie (perro/gato), sexo, raza, rango de edad, distancia máxima, esterilización, vacunas, padecimientos, y criterio de ordenamiento.
4. El usuario presiona "Aplicar filtros".
5. El sistema envía la solicitud al backend con los filtros seleccionados.
6. El backend valida la autenticación y el rol del usuario.
7. El backend aplica los filtros sobre la base de datos y en memoria según corresponda, y devuelve la lista resultante.
8. El frontend actualiza el catálogo mostrando únicamente los animales que cumplen los criterios.
9. El usuario puede además buscar por nombre o raza directamente en el campo de búsqueda, lo cual filtra localmente sobre los resultados ya cargados.

Flujos Alternativos:

- A1 – Ningún animal coincide con los filtros: El sistema devuelve una lista vacía. El frontend muestra el mensaje "No se encontraron mascotas con esos filtros" y ofrece la opción de limpiar los filtros.
- A2 – El usuario limpia los filtros: El sistema restaura el catálogo completo sin restricciones.

Postcondiciones:

- El catálogo muestra únicamente los animales que cumplen los criterios seleccionados.
- El usuario puede acceder a la ficha de cualquier animal del resultado (CU14).

CU16 - Marcar publicación como inapropiada (RF17)

Actores: Usuario autenticado con rol ADOPTANTE

Descripción: Este caso de uso describe el proceso mediante el cual un usuario reporta una publicación de animal como inapropiada, proporcionando un motivo para su revisión por un administrador.

Precondiciones:

- El usuario está autenticado.
- La publicación del animal existe en el sistema.
- El usuario no ha reportado previamente el mismo animal.

Flujo Principal:

1. El usuario accede a la publicación de un animal.
2. El usuario selecciona la opción "Reportar".
3. El sistema muestra un campo para ingresar el motivo del reporte.
4. El usuario ingresa el motivo y confirma.
5. El sistema valida la autenticación del usuario.
6. El sistema valida que el animal exista.
7. El sistema verifica que el usuario no haya reportado previamente el mismo animal.
8. El sistema registra el reporte con estado PENDIENTE en la base de datos.
9. El sistema confirma que la publicación fue reportada exitosamente.

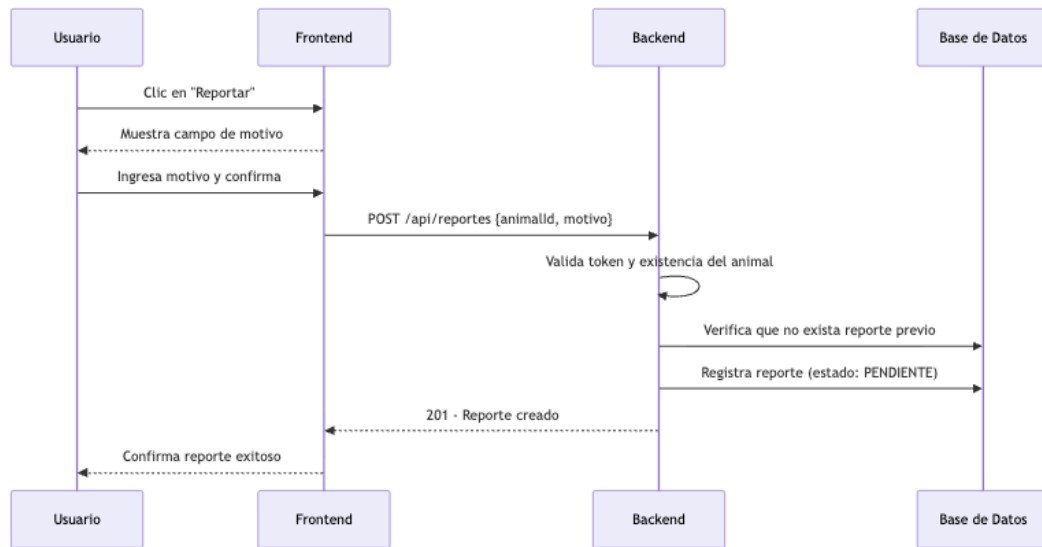
Flujos Alternativos:

- A1 – Usuario no autenticado: El sistema devuelve un error 401 y rechaza la solicitud.
- A2 – Animal no encontrado: El sistema devuelve un error 404.
- A3 – Usuario ya reportó este animal: El sistema devuelve un error 400 indicando que ya existe un reporte.
- A4 – Motivo vacío: El sistema devuelve error 400 "motivo es requerido."

Postcondiciones:

- La publicación queda con un reporte PENDIENTE asociado para revisión por un administrador.

Diagrama:



CU17 - Notificación de ingreso exitoso (RF14)

Actores:

Usuario registrado

Sistema de correo electrónico

Descripción: Este caso de uso describe el proceso mediante el cual el sistema notifica automáticamente al usuario cuando completa un inicio de sesión exitoso (tras 2FA).

Precondiciones:

- El usuario completó ambos factores de autenticación exitosamente.

Flujo Principal:

1. El usuario completa la verificación 2FA exitosamente.
2. El sistema genera el token de sesión.
3. El sistema envía automáticamente un correo de notificación al correo registrado del usuario.

4. El sistema registra la acción de login en el log de actividad.
5. El sistema retorna el token al frontend.

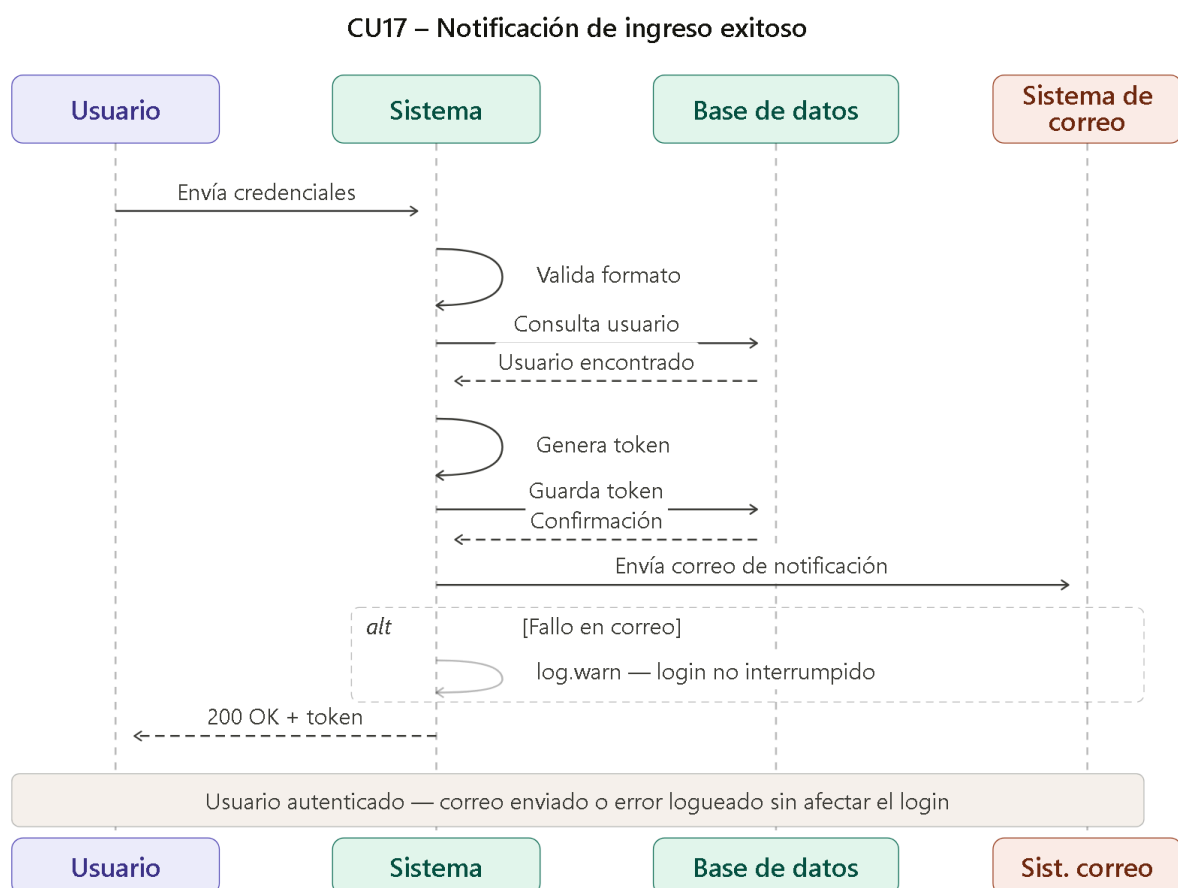
Flujos Alternativos:

- A1 — Fallo en el envío del correo: El sistema registra el error en los logs pero no interrumpe el login. El usuario queda autenticado igualmente.

Postcondiciones:

- El usuario queda autenticado en el sistema.
- El usuario recibe (o intenta recibir) un correo informando del nuevo acceso.
- La acción queda registrada en el log de actividad.

Diagrama:



CU18 - Eliminación de cuenta (RF15)

Actores: Usuario autenticado, Sistema de correo electrónico

Descripción: Este caso de uso describe el proceso mediante el cual un usuario solicita la eliminación lógica de su propia cuenta dentro de la plataforma.

Precondiciones:

- El usuario está autenticado.
- La cuenta no ha sido eliminada previamente.

Flujo Principal:

1. El usuario accede a la vista de perfil y selecciona la opción "Eliminar mi cuenta".
2. El sistema solicita confirmación al usuario.
3. El usuario confirma la eliminación.
4. El sistema recibe la solicitud con el token de autenticación.
5. El sistema valida el token y verifica que la cuenta no esté ya eliminada.
6. El sistema establece la fecha_eliminado e invalida el token.
7. El sistema envía un correo de confirmación al usuario.
8. El frontend elimina el token del sessionStorage y redirige al login.

Flujos Alternativos:

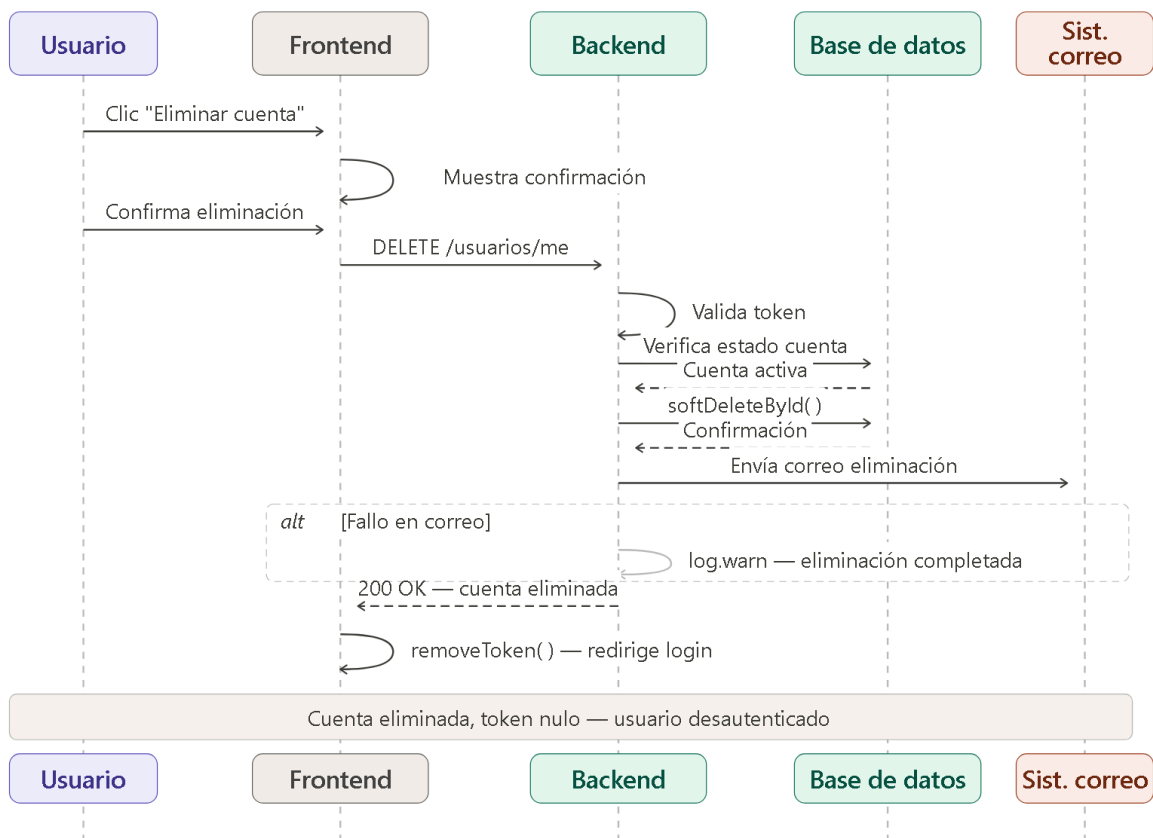
- A1 — Token ausente o inválido: El sistema devuelve error 401 y rechaza la solicitud.
- A2 — Cuenta ya eliminada: El sistema devuelve error 400 indicando que la cuenta ya fue eliminada.
- A3 — Fallo en el correo de confirmación: El sistema registra el error en logs pero la eliminación se completa igualmente.
- A4 — Usuario cancela la confirmación: El sistema no realiza ningún cambio.

Postcondiciones:

- La cuenta queda con fecha_eliminado establecida y token nulo.
- El usuario no puede volver a iniciar sesión.
- El usuario recibe correo de confirmación.

Diagrama:

CU18 – Eliminación de cuenta



CU19 - Historial de animales adoptados (RF16)

Actores: Cuidador autenticado

Descripción: Este caso de uso describe el proceso mediante el cual un cuidador consulta la lista de sus animales que han sido marcados como adoptados.

Precondiciones:

- El usuario está autenticado con rol CUIDADOR.

Flujo Principal:

1. El cuidador accede a la vista "Mis mascotas".
2. El sistema recibe la solicitud con el token de autenticación.
3. El sistema valida el token y verifica que el usuario tenga rol CUIDADOR.
4. El sistema consulta todos los animales del cuidador con estatus ADOPTADO.

5. El sistema retorna la lista con información básica de cada animal (nombre, especie, raza, foto de portada).
6. El frontend muestra la sección "Historial de adoptados" con las tarjetas correspondientes.

Flujos Alternativos:

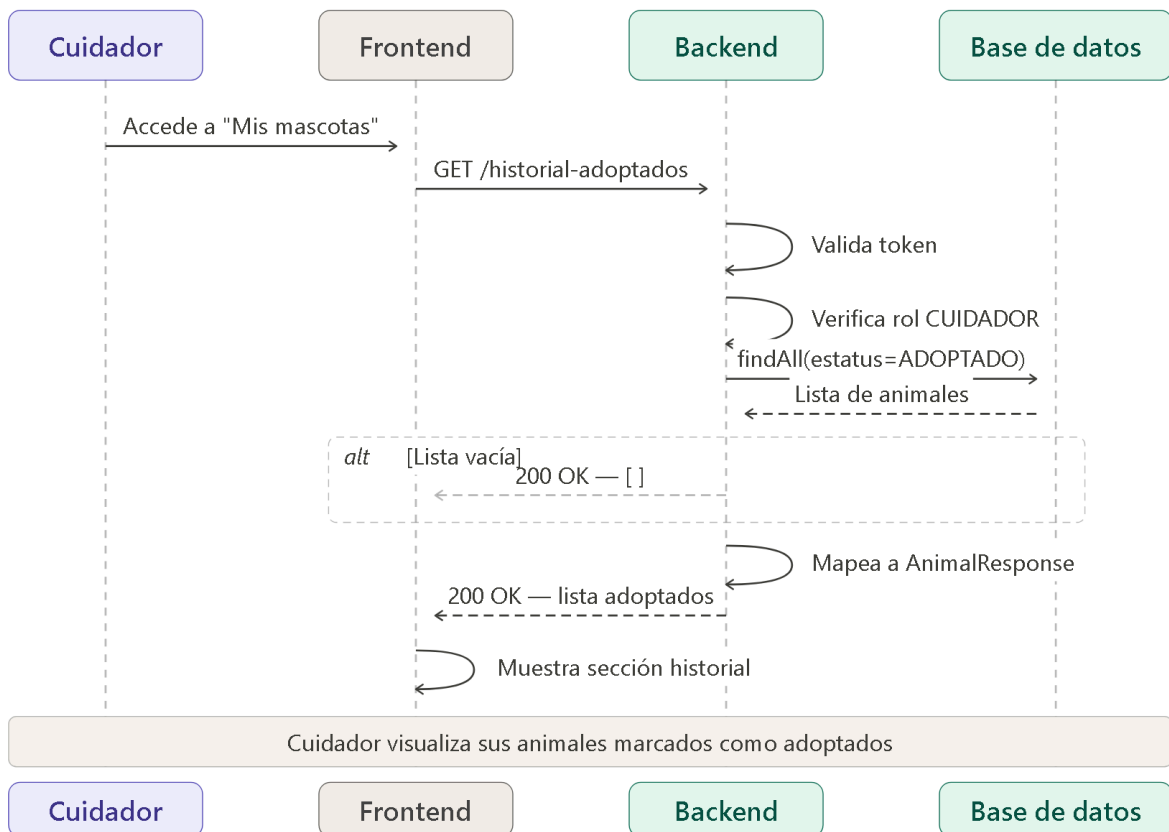
- A1 — Token ausente o inválido: El sistema devuelve error 401. El frontend redirige al login.
- A2 — Usuario no es CUIDADOR: El sistema devuelve error 400 con mensaje "Solo los cuidadores pueden consultar su historial de adoptados."
- A3 — Lista vacía: El sistema retorna lista vacía. El frontend muestra mensaje "Aún no tienes animales marcados como adoptados."

Postcondiciones:

- El cuidador visualiza su historial de animales que han encontrado hogar.

Diagrama:

CU19 – Historial de animales adoptados



CU20 – Verificación de código 2FA (RF18)

Actores:

- Usuario registrado
- Sistema de correo electrónico

Descripción: Este caso de uso describe el proceso mediante el cual el sistema solicita y valida un código de verificación de segundo factor tras el ingreso exitoso de credenciales.

Precondiciones:

- El usuario proporcionó credenciales válidas.
- El correo del usuario está verificado.
- La cuenta no está bloqueada.

Flujo Principal:

1. El sistema valida las credenciales del usuario.

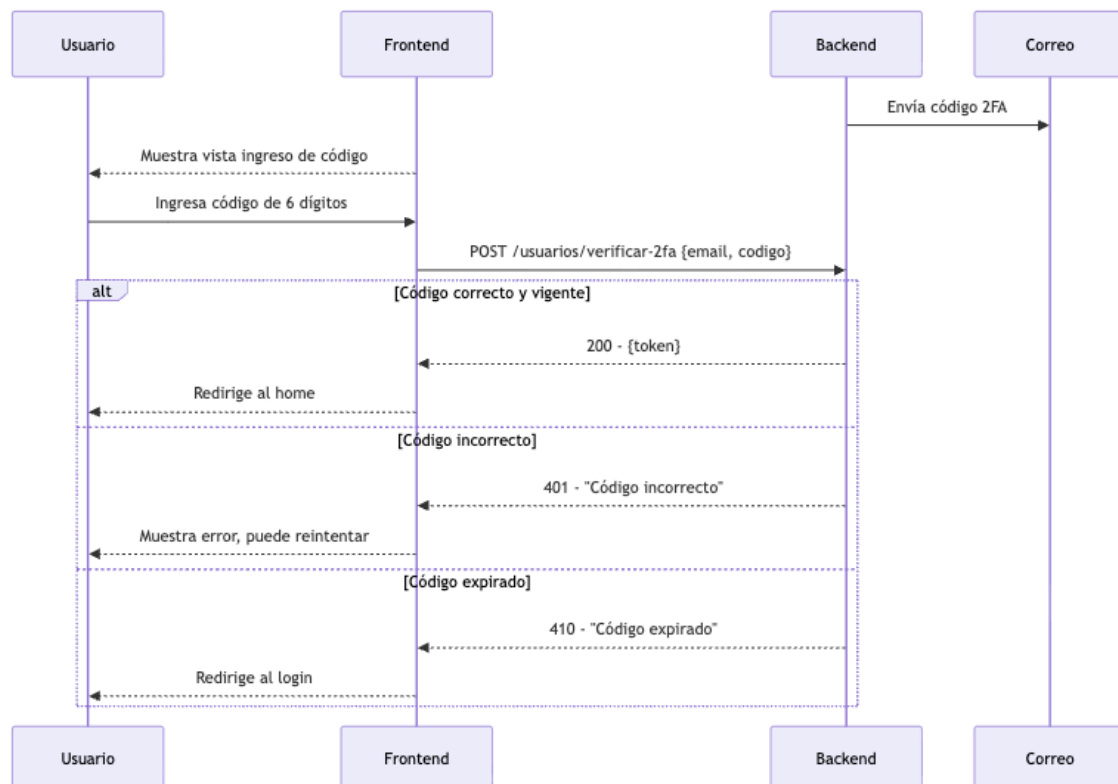
2. El sistema genera un código numérico de 6 dígitos con expiración de 10 minutos.
3. El sistema envía el código al correo registrado del usuario.
4. El frontend muestra la vista de ingreso de código 2FA.
5. El usuario ingresa el código recibido.
6. El sistema valida que el código sea correcto y no haya expirado.
7. El sistema genera el token de sesión y lo retorna al frontend.
8. El frontend almacena el token y redirige al home.

Flujos Alternativos:

- A1 – Código incorrecto: El sistema devuelve error 401 con mensaje "Código incorrecto." El usuario puede reintentar.
- A2 – Código expirado: El sistema devuelve error 410 con mensaje "El código ha expirado. Inicia sesión de nuevo." El usuario debe repetir el proceso desde el login.

Postcondiciones:

- El usuario queda autenticado con token de sesión válido.



CU21 – Verificación de correo electrónico (RF19)

Actores: Usuario nuevo, Sistema de correo electrónico

Descripción: Este caso de uso describe el proceso mediante el cual un usuario recién registrado verifica la propiedad de su correo electrónico para activar su cuenta.

Precondiciones:

- El usuario completó el registro exitosamente.
- La cuenta está creada con `verificado = false`.

Flujo Principal:

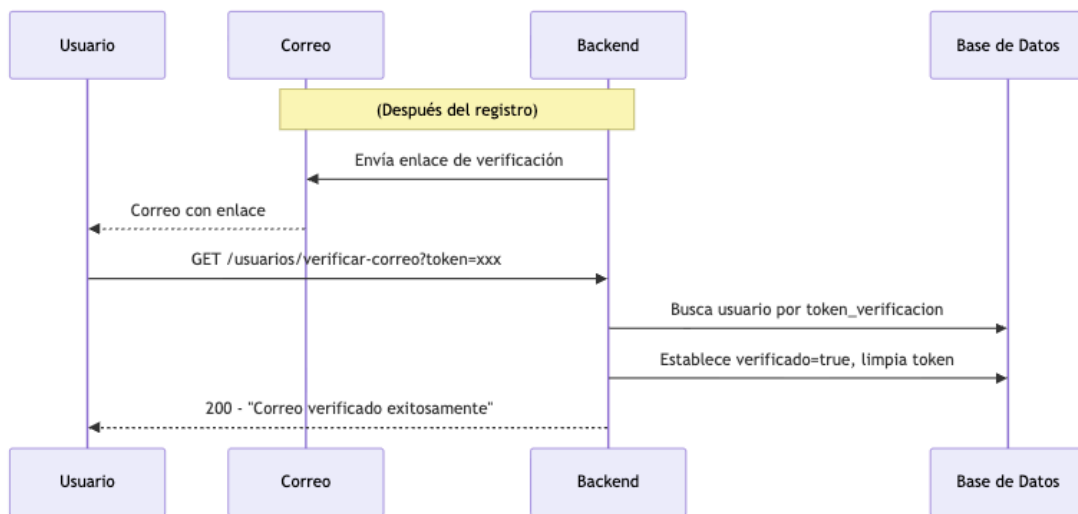
1. Al completar el registro, el sistema genera un token de verificación único.
2. El sistema envía un correo al usuario con un enlace que contiene el token.
3. El usuario accede al enlace desde su bandeja de entrada.
4. El sistema recibe la solicitud GET con el token de verificación.
5. El sistema valida el token y marca la cuenta como verificada (`verificado = true`).
6. El sistema confirma "Correo verificado exitosamente. Ya puedes iniciar sesión."

Flujos Alternativos:

- A1 – Token inválido o inexistente: El sistema devuelve error 400 "Token de verificación inválido."
- A2 – Cuenta ya verificada: El sistema responde indicando que la cuenta ya está activa.

Postcondiciones:

- La cuenta queda con `verificado = true`.
- El usuario puede iniciar sesión.



CU22 – Recuperación de contraseña (RF20)

Actores:

- Usuario registrado
- Sistema de correo electrónico

Descripción: Este caso de uso describe el proceso mediante el cual un usuario que olvidó su contraseña solicita un enlace de restablecimiento por correo.

Precondiciones:

- El usuario tiene una cuenta registrada.

Flujo Principal:

1. El usuario accede a la vista de recuperación de contraseña.
2. El usuario ingresa su correo electrónico y envía la solicitud.
3. El sistema genera un token de recuperación y lo asocia al usuario (si el correo existe).
4. El sistema envía un correo con enlace de restablecimiento (si el correo existe).
5. El sistema responde con mensaje genérico "Si el correo existe, recibirás un enlace para restablecer tu contraseña."
6. El usuario accede al enlace desde su correo y es dirigido a la vista de nueva contraseña.

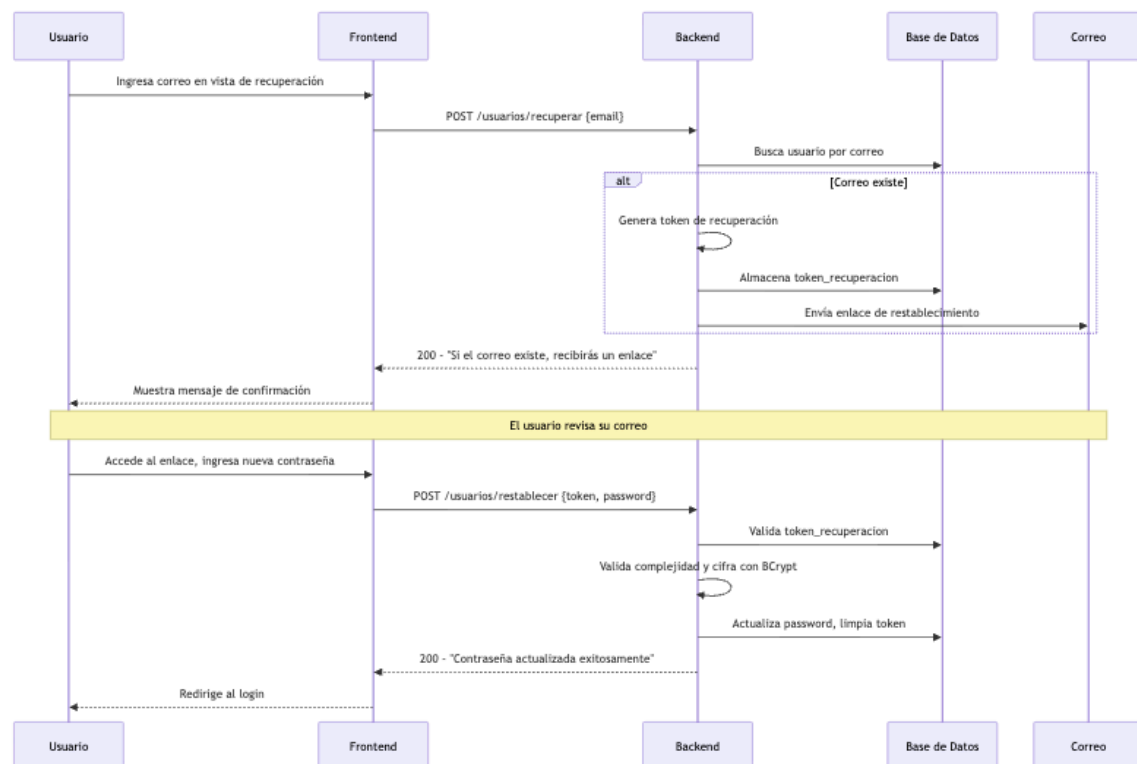
7. El usuario ingresa su nueva contraseña (cumpliendo requisitos de complejidad).
8. El sistema valida el token de recuperación y actualiza la contraseña (cifrada con BCrypt).
9. El sistema confirma "Contraseña actualizada exitosamente."

Flujos Alternativos:

- A1 – Correo no registrado: El sistema responde con el mismo mensaje genérico sin revelar información. No se envía correo.
- A2 – Token de recuperación inválido: El sistema devuelve error 400 "Token inválido o expirado."
- A3 – Contraseña no cumple requisitos: El sistema devuelve error 400 indicando los criterios faltantes.

Postcondiciones:

- La contraseña queda actualizada.
- El usuario puede iniciar sesión con la nueva contraseña.



CU23 – Gestión de reportes por administrador (RF22)

Actores: Administrador autenticado, Sistema de correo electrónico

Descripción: Este caso de uso describe el proceso mediante el cual un administrador revisa y gestiona los reportes de publicaciones inapropiadas.

Precondiciones:

- El usuario está autenticado con rol ADMINISTRADOR.
- Existen reportes con estado PENDIENTE.

Flujo Principal:

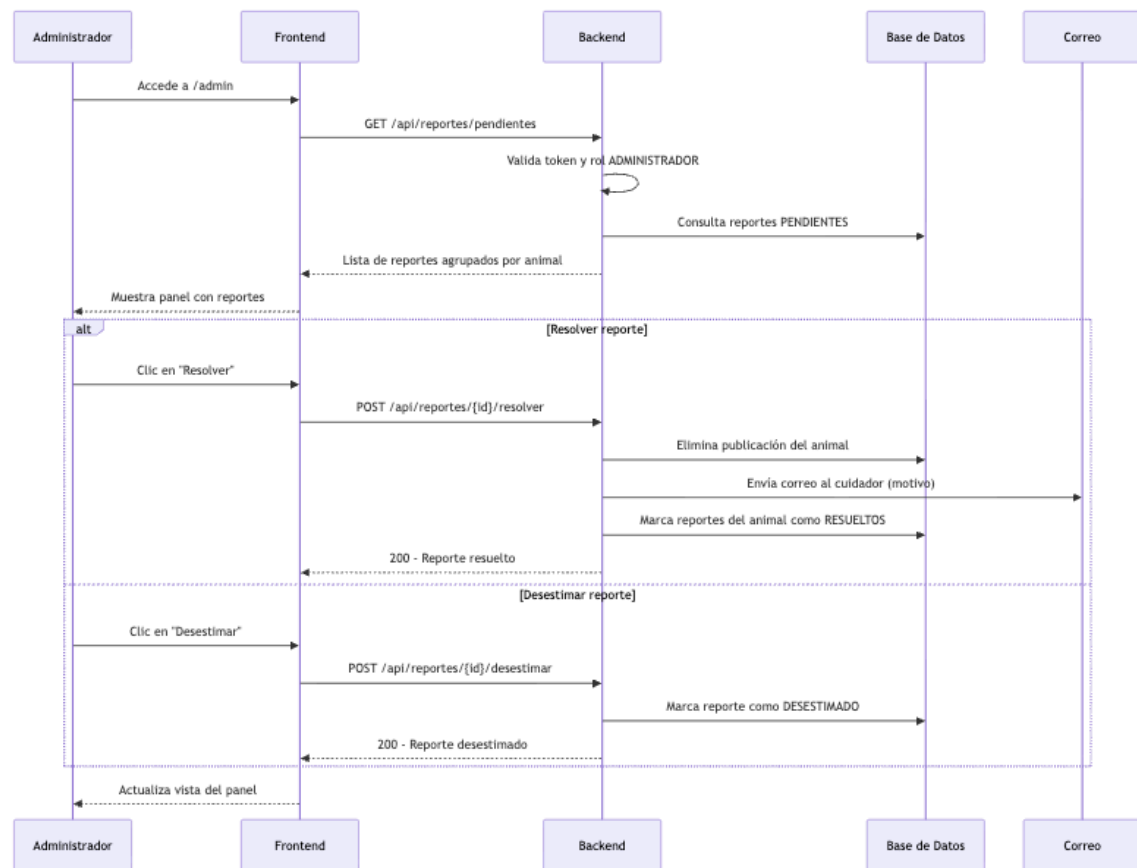
1. El administrador accede al panel de administración (/admin).
2. El sistema valida el token y el rol ADMINISTRADOR.
3. El sistema muestra la lista de reportes pendientes agrupados por publicación, ordenados por cantidad de reportes (mayor primero).
4. El administrador selecciona una acción sobre un grupo de reportes:
 - **Resolver:** El sistema elimina la publicación del animal, envía un correo al cuidador informando la eliminación y el motivo, y marca todos los reportes de ese animal como RESUELTOS con fecha de resolución.
 - **Desestimar** El sistema marca los reportes como DESESTIMADOS sin afectar la publicación.
5. El sistema confirma la acción y actualiza la vista.

Flujos Alternativos:

- A1 – Token ausente o inválido: El sistema devuelve error 401.
- A2 – Usuario no es ADMINISTRADOR: El sistema devuelve error 403 "Acceso denegado." El frontend redirige al home.
- A3 – Reporte no encontrado: El sistema devuelve error 400.

Postcondiciones:

- El reporte queda con estado RESUELTO o DESESTIMADO con fecha de resolución.
- Si fue resuelto: la publicación es eliminada y el cuidador recibe notificación por correo con el motivo.



5. Requerimientos No Funcionales

Tipo		Descripción	Criterios de aceptación	RF
Seguridad	RNF1	Almacenamiento seguro de contraseñas. Las contraseñas deberán almacenarse mediante un algoritmo de cifrado seguro.	Las contraseñas de los usuarios deberán almacenarse de forma irreversible mediante un algoritmo de cifrado unidireccional. En ningún caso se almacenarán en texto plano.	RF1
	RNF2	Ninguna interfaz o funcionalidad del sistema estará disponible para usuarios no autenticados.	El sistema deberá requerir autenticación para acceder a endpoints protegidos. Las solicitudes sin autenticación	RF2

			válida deberán ser rechazadas. El sistema no deberá permitir acceso a información sin autenticación. Los intentos a cualquier URL del sistema sin autenticación previa deberán redirigir al usuario a la página de inicio de sesión. Usuarios no autenticados serán redirigidos a la página de inicio de sesión.	
Accesibilidad y Usabilidad	RNF5	Mostrar errores claros y comprensibles	El sistema debe explicar al usuario que salió mal y qué acción debe tomar para corregirlo. Se debe tener mensajes para errores de formato en registro, errores de inicio de sesión, errores de red, etc.	
		Interfaz atractiva y usable	Los usuarios nuevos deben poder completar una muestra de interés de animal sin asistencia externa en menos de 2 minutos.	
		Interfaz adaptable (responsive)	Debe poder mantener su usabilidad en celular, tablet y escritorio	
		Fácil de usar	El registro de un animal no deberá requerir más de 5 interacciones del usuario (clics o cambios de pantalla).	
Privacidad	RNF6	No mostrar información personal a terceros	La información mostrada a terceros (como código postal) no se pueden usar para identificar de manera única La información personal como curp, nombre completo y contraseña se almacenarán con estricta seguridad y no será disponible para nadie fuera de	RF4, RF5

			los administradores del sistema y el propio usuario La comunicación inicial entre usuarios deberá realizarse únicamente mediante el sistema de notificación por correo automático.	
		Privacidad de datos de contacto: Los datos de contacto de los usuarios solo deberán compartirse a través del mecanismo de notificación definido por el sistema. La plataforma no deberá exponer esta información de forma pública ni accesible para terceros no involucrados.	Solo a través del sistema de correo electrónico es que los datos de correo de los usuarios se compartirán.	
Mantenibilidad	RNF7	El código debe ser modular con una clara separación de capas	Utiliza MVC adaptado a servicios REST	
	RNF8	Pruebas	Incluye pruebas en postman para validar el funcionamiento	
Trazabilidad de acciones	RNF9	El sistema deberá mantener un registro interno de acciones críticas (login, eliminación de cuenta, resolución de reportes) con identificador de usuario y marca de tiempo, sin almacenar PII.	El log persiste en tabla `accion`. No es accesible desde la interfaz de usuario. Se registra al menos: usuario_id, descripción del evento y fecha.	

6. Tecnologías Utilizadas

a) Lenguaje de programación en back-end: **Kotlin**

Kotlin fue el lenguaje de programación escogido para el back-end durante los laboratorios de la materia, Kotlin nos brinda compatibilidad con java y podemos usar el ecosistema de bibliotecas de Java, además de tener una sintaxis más clara que otros lenguajes de programación de JVM lo cual nos permite hacer mejores aplicaciones, robustas, seguras y fáciles de mantener; es compatible con frameworks modernos y podemos desarrollar APIs REST.

b) Framework en back-end: **Spring Boot**

Este es el framework que utilizaremos para el back-end, nos ayuda a la creación de aplicaciones REST que sean modulares y podemos tener una arquitectura en capas, además de tener soporte nativo y cuenta con una amplia documentación.

c) Base de datos: **Postgres**

Elegimos esta base de datos dado que es robusta y que gran parte del equipo ya tenía familiaridad con la misma, por lo que eso nos facilita el trabajar todos con ella. Además de ser estable, nos brinda confiabilidad, ya que maneja los datos de manera segura.

d) **Hibernate:**

Permite la interacción entre el sistema y la base de datos de manera más eficiente, facilita la conversión de objetos del lenguaje de programación a registros en la base de datos.

e) **Postman:**

Herramienta de prueba para los endpoints, envía solicitudes HTTP y verifica las respuestas del sistema.

f) **Git:**

Sistema de control de versiones, el cual nos ayuda a tener un seguimiento de los cambios en el sistema y mantener las versiones estables.

g) Frontend: **NextJS**

Se utilizó Next.js como framework para el desarrollo del frontend debido a su capacidad para construir aplicaciones modernas basadas en **React**, así como su soporte para renderizado eficiente y organización estructurada del proyecto.

h) Framework en front-end: **TailwindCSS**

Se utilizó Tailwind CSS como framework de estilos para agilizar el desarrollo de interfaces consistentes sin necesidad de escribir grandes cantidades de CSS personalizado.

- Además, se utilizó **DaisyUI** como complemento de Tailwind CSS para proporcionar componentes visuales predefinidos

i) Lenguaje de programación en front-end: **TypeScript**

Se utilizó TypeScript como lenguaje de programación en el front-end, el cual extiende JavaScript incorporando tipado estático. Esto permite mejorar la calidad del código y facilitar el mantenimiento del sistema.

El uso de TypeScript contribuye a una mayor robustez en la aplicación, especialmente en la interacción con la API del back-end, al definir estructuras claras para los datos enviados y recibidos.

j) Mecanismo de comunicación con back-end: **Axios**

Se utilizó Axios como biblioteca para la comunicación con el back-end mediante HTTP. Permite centralizar la configuración de las peticiones, manejar respuestas de forma sencilla y aplicar interceptores para la autenticación, evitando la repetición de código en cada llamada.

k) **Cloudinary**: Servicio externo de almacenamiento y optimización de imágenes. Se utiliza para guardar las fotos de perfil de los usuarios y las fotografías de los animales publicados. Proporciona transformaciones automáticas de formato, calidad y dimensiones mediante su SDK para Java (cloudinary-http45) en el backend y @cloudinary/url-gen + @cloudinary/react en el frontend.

l) **Spring Security Crypto (BCrypt)**: Librería del ecosistema Spring utilizada para el cifrado seguro de contraseñas mediante el algoritmo BCrypt. Reemplaza el uso de SHA-256 mencionado en versiones anteriores del documento. BCrypt incluye salt automático, lo que garantiza que contraseñas idénticas produzcan hashes distintos.

m) **Spring Boot Mail / JavaMailSender + Gmail SMTP**: Para mandar correos electrónicos desde la cuenta de Colitas Felices de Gmail.

- n) **Cally**: Web component de código abierto para selección de fechas. Se utiliza en los formularios de publicación y edición de animales. Es compatible con cualquier framework al ser un custom element nativo del browser, y daisyUI lo estiliza automáticamente.
- o) **dotenv-kotlin**: Librería para cargar variables de entorno desde un archivo .env en el backend. Permite separar la configuración sensible (credenciales de base de datos, correo, Cloudinary) del código fuente.
- p) **Nominatim**: Servicio de geocodificación basado en los datos abiertos de OpenStreetMap Foundation. Se utiliza para obtener coordenadas aproximadas (latitud y longitud) a partir de códigos postales mediante la API pública <https://nominatim.openstreetmap.org/search> (ejemplo de uso: <https://nominatim.openstreetmap.org/search?postalcode=146401&country=Mexico&format=json>). Los datos provienen de contribuciones colaborativas de usuarios, y servicios provistos por terceros. El servicio es gratuito para usos moderados, aunque cuenta con límites de uso (máximo 1 request por segundo). En el sistema, las coordenadas obtenidas se almacenan en la base de datos para evitar consultas repetidas y reducir la dependencia de la API externa.
- q) **The Dog API**: API pública que proporciona información detallada sobre razas de perros, incluyendo características, temperamento e imágenes representativas. Se utiliza para enriquecer las publicaciones de perros con datos adicionales sobre su raza.
- r) **The Cat API**: API pública equivalente a The Dog API pero orientada a razas de gatos. Proporciona información sobre características y temperamento de las razas felinas registradas en el sistema.
- s) **Google Translate API**: Servicio de traducción automática de Google. Se utiliza para traducir al español las descripciones de razas obtenidas desde The Dog API y The Cat API, las cuales retornan información en inglés.
- t) **Leaflet**: Biblioteca de código abierto para mapas interactivos. Se utiliza en el frontend para mostrar un mapa con la ubicación aproximada de los animales disponibles, basada en las coordenadas del código postal del cuidador. Implementa clustering de

marcadores para agrupar publicaciones cercanas y muestra la foto de portada del animal en cada marcador.

u) **Zxcvbn**: Biblioteca utilizada para estimar la seguridad de una contraseña.

7. Arquitectura General

Patrón utilizado: **MVC adaptado a servicios REST**

Este patrón de diseño está implementado mediante una estructura en capas (Modelo, Vista, Controlador), lo cual nos permite una mejor separación de responsabilidades, facilita el mantenimiento y el desarrollo colaborativo. Pero a partir de la Iteración 2, el sistema funciona como un conjunto de dos componentes independientes que solo se comunican entre sí.

Arquitectura en capas:

a) **Controller:**

Es el responsable de recibir las peticiones HTTP del cliente, valida los datos, usa los servicios correspondientes y devuelve la respuesta en formato JSON, este es el intermediario entre el usuario y el interior de la aplicación.

b) **Service:**

Aquí se encuentra el comportamiento de nuestra aplicación; procesa la información mandada por el controller, aplica las reglas de negocio y gestiona procesos.

c) **Repository:**

Es la capa encargada del manejo de la base de datos, facilita la modificación futura del sistema de almacenamiento si fuera necesario, realiza consultas, guarda datos, recupera y actualiza datos.

Componentes del sistema:

a) **Frontend:**

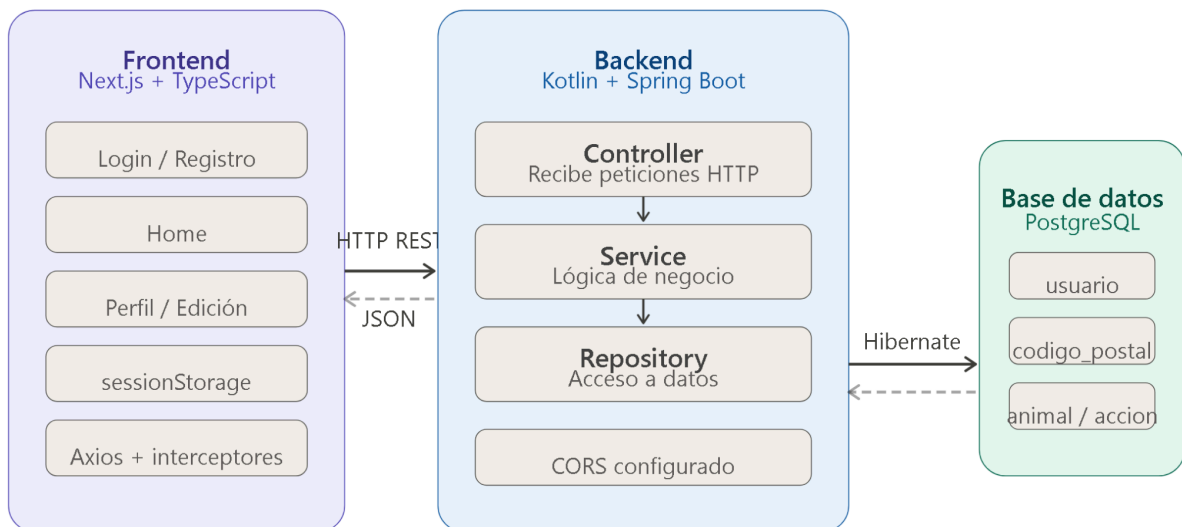
Es la interfaz destinada para el usuario y esta fue desarrollada con Next.js que utiliza la API del backend por medio de peticiones HTTP, la cual puede mostrar las vistas al Usuario, gestionar la sesión activa en el navegador y proteger las rutas que requieran autenticación; de esta forma coordina la interacción del usuario con el backend.

b) **Backend:**

API REST se organiza en tres capas; una de ellas es el Controller, el cual recibe las peticiones HTTP del frontend, validando los datos y devolviéndolos como respuestas en formato JSON. Además, Service genera tokens, valida las credenciales y aplica las reglas del sistema, donde Repository es nuestra capa de acceso a los datos (como la consulta, la actualización, registros), que se guardan en la base de datos mediante Hibernate.

c) **Comunicación:**

El frontend realiza peticiones HTTP al backend utilizando Axios, en donde cada petición que requiera alguna autenticación viene con un token en el encabezado Authorization: Bearer <token>. Además, el backend tiene CORS configurado para poder aceptar peticiones del frontend y obtener una respuesta en formato JSON.



Patrones de diseño aplicados:

- **Builder (EmailBuilder):** Se utiliza para construir correos HTML de forma legible y paso a paso, permitiendo definir saludo, líneas de contenido y botones de acción de manera encadenada. Facilita la construcción de objetos complejos (correos con estructura HTML) sin exponer la complejidad interna del formato.
- **Factory Method (NotificacionFactory):** Centraliza la creación del contenido de las notificaciones por correo según el tipo de evento (interés manifestado, interés retirado, código 2FA, verificación de correo, recuperación de contraseña, publicación eliminada, cuenta eliminada). Cada método produce un par (asunto, cuerpo HTML) listo para enviar.
- **Strategy (PasswordValidator):** Implementa la validación de contraseñas como un conjunto de reglas independientes (longitud mínima, mayúscula, minúscula, dígito, carácter especial). Cada regla es una estrategia que puede agregarse o eliminarse sin modificar el validador principal, cumpliendo con el principio abierto/cerrado.

8. Flujo de Autenticación

El sistema implementa un flujo de autenticación en dos factores basado en tokens para gestionar la sesión de los usuarios:

1. El usuario ingresa sus credenciales (correo y contraseña) en la interfaz de login.
2. El frontend envía una solicitud al backend.
3. El backend verifica que la cuenta no esté bloqueada y que el correo esté verificado.
4. El backend valida las credenciales (comparación BCrypt).
5. Si son correctas, el backend genera un código 2FA de 6 dígitos, lo almacena con expiración de 10 minutos y lo envía por correo.
6. El frontend muestra la vista de ingreso de código 2FA.
7. El usuario ingresa el código recibido por correo.
8. El backend valida el código y, si es correcto, genera un token UUID de sesión.
9. El frontend almacena el token en sessionStorage.
10. En cada solicitud posterior, el frontend envía el token en el header `Authorization: Bearer <token>`.

11. El backend valida el token antes de permitir acceso a endpoints protegidos.

En caso de fallo:

- Credenciales incorrectas: Se incrementa el contador de intentos fallidos. Tras superar el límite, la cuenta se bloquea 15 minutos.
- Código 2FA incorrecto: Se rechaza sin otorgar token. El usuario puede reintentar.
- Código 2FA expirado: El usuario debe reiniciar el proceso de login.
- Token inexistente o inválido en solicitudes posteriores: El backend rechaza con error 401 y el frontend redirige al login.

9. Modelo Conceptual de Datos

Este sistema se basa en la gestión de varias entidades interrelacionadas que permiten modelar el flujo completo de adopción dentro de la plataforma.

Entidad: Usuario

Esta entidad representa a una persona registrada en el sistema que puede interactuar con la aplicación según su rol asignado.

Atributos principales:

- IDUsuario: UUID. Identificador único de un usuario dentro del sistema. Llave primaria.
- curp: VARCHAR(18). Clave Única de Registro de Población, utilizada para garantizar que cada persona física solo esté asociada a una cuenta.
- username: VARCHAR(50). Nombre con el que se identifica el usuario en su perfil y al interactuar con otros.
- rol: rol_enum con valores ADOPTANTE, CUIDADOR, ADMINISTRADOR. Define el tipo de usuario y los comportamientos disponibles dentro del sistema.
- foto_perfil: VARCHAR(255). URL de Cloudinary que almacena la foto del usuario.
- nombres: VARCHAR(100). Nombre(s) real(es) del usuario registrado.

- apellido_paterno: VARCHAR(100). Apellido paterno del usuario.
- apellido_materno: VARCHAR(100). Apellido materno del usuario.
- email: VARCHAR(100). Dirección de correo electrónico utilizada como identificador de inicio de sesión y medio de notificación.
- codigo_postal: VARCHAR(5). Código postal que permite conocer la ubicación aproximada del usuario sin exponer su dirección exacta.
- password: VARCHAR(255). Contraseña cifrada del usuario.
- token: VARCHAR(255). Token UUID de sesión activa del usuario, generado al iniciar sesión.
- fecha_registro: TIMESTAMP. Fecha y hora en que se creó la cuenta.
- fecha_update: TIMESTAMP. Fecha y hora de la última actualización del perfil.
- fecha_eliminado: TIMESTAMP. Fecha y hora en que se eliminó el perfil (soft delete, reservado para iteraciones futuras).
- codigo_2fa: VARCHAR(6). Código de verificación de segundo factor, generado en cada inicio de sesión.
- codigo_2fa_expira: TIMESTAMP. Fecha y hora de expiración del código 2FA.
- intentos_fallidos: INT. Contador de intentos fallidos consecutivos de login (default 0).
- bloqueado_hasta: TIMESTAMP. Fecha hasta la cual la cuenta está bloqueada temporalmente (null si no está bloqueada).
- verificado: BOOLEAN. Indica si el correo electrónico del usuario ha sido verificado (default false).
- token_verificacion: TEXT. Token generado para verificar el correo al registrarse.
- token_recuperacion: TEXT. Token generado para restablecer la contraseña.

- email_pendiente: VARCHAR(255). Nuevo correo electrónico pendiente de verificación tras solicitud de cambio (null si no hay cambio pendiente).

Entidad: Animal

Esta entidad representa a un animal registrado en la plataforma por un cuidador, disponible para adopción.

Atributos principales:

- IDAnimal: UUID. Identificador único del animal dentro del sistema. Llave primaria.
- nombre: VARCHAR(100). Nombre del animal.
- descripcion: TEXT. Descripción general del animal.
- tipo: tipo_enum con valores PERRO, GATO. Tipo de animal.
- raza: VARCHAR(100). Raza o raza similar del animal.
- fecha_nacimiento: DATE. Fecha de nacimiento del animal, a partir de la cual se calcula la edad en la interfaz.
- genero: genero_enum con valores MACHO, HEMBRA. Género del animal.
- esterilizado: BOOLEAN. Indica si el animal está esterilizado.
- estatus: estatus_enum con valores DISPONIBLE, ADOPTADO. Estado actual de la publicación.
- fecha_publicacion: TIMESTAMP. Fecha y hora en que se publicó el animal.
- fecha_update: TIMESTAMP. Fecha y hora de la última actualización de la ficha.
- id_cuidador: UUID (FK). Referencia al usuario cuidador que publicó el animal. La ubicación del animal se infiere del código postal del cuidador asociado.
- vacunas: Relación N:M con la entidad Vacuna a través de la tabla animal_vacuna. Cada vacuna tiene un nombre único en el catálogo del sistema.

- padecimientos: Relación N:M con la entidad Padecimiento a través de la tabla animal_padecimiento. Cada padecimiento tiene un nombre único en el catálogo del sistema.
- fotos: Relación 1:N con la entidad FotoAnimal. Cada foto almacena una URL de Cloudinary. Un animal puede tener múltiples fotos.

Entidad: Vacuna

Esta entidad representa una vacuna dentro del catálogo del sistema.

Atributos principales:

- vacuna_id: UUID. Identificador único de la vacuna. Llave primaria.
- nombre: VARCHAR(255). Nombre de la vacuna.

Entidad: Padecimiento

Esta entidad representa un padecimiento o condición de salud dentro del catálogo del sistema.

Atributos principales:

- padecimiento_id: UUID. Identificador único del padecimiento. Llave primaria.
- nombre: VARCHAR(255). Nombre del padecimiento.

Entidad: FotoAnimal

Esta entidad almacena las fotografías asociadas a un animal.

Atributos principales:

- foto_id: UUID. Identificador único de la foto. Llave primaria.
- animal_id: UUID (FK). Referencia al animal al que pertenece la foto.
- foto: TEXT. URL de Cloudinary donde se almacena la imagen.
- fecha: TIMESTAMP. Fecha y hora en que se subió la foto.

Entidad: Interés

Esta entidad representa la manifestación de interés de un adoptante en un animal específico.

Atributos principales:

- `id_adoptante`: UUID (FK). Referencia al usuario adoptante que manifestó interés.
- `id_animal`: UUID (FK). Referencia al animal en el que se manifestó interés.
- `fecha_interes`: TIMESTAMP. Fecha y hora en que se registró la manifestación de interés.

Llave primaria compuesta: (`id_adoptante`, `id_animal`). No existe un identificador UUID independiente para esta entidad.

Entidad:CodigoPostal

Esta entidad almacena las coordenadas geográficas asociadas a códigos postales para calcular distancias entre usuarios y animales.

Atributos principales:

- `codigo_postal`: VARCHAR(5). Código postal. Llave primaria.
- `latitud`: DECIMAL(10,6). Latitud geográfica.
- `longitud`: DECIMAL(10,6). Longitud geográfica.

Entidad: Raza

Esta entidad almacena el catálogo de razas de perros y gatos con sus nombres en español e inglés para la integración con APIs externas.

Atributos principales:

- `raza_id`: UUID. Identificador único. Llave primaria.
- `especie`: VARCHAR(10). Tipo de animal (PERRO o GATO).
- `nombre_es`: VARCHAR(100). Nombre de la raza en español.
- `nombre_en`: VARCHAR(100). Nombre de la raza en inglés (utilizado para consultas a APIs externas).

- Restricción UNIQUE: (especie, nombre_en).

Entidad: Reporte

Esta entidad representa un reporte de publicación inapropiada realizado por un usuario.

Atributos principales:

- reporte_id: UUID. Identificador único. Llave primaria.
- usuario_id: UUID (FK). Usuario que realizó el reporte.
- animal_id: UUID (FK). Animal reportado.
- motivo: TEXT. Razón del reporte proporcionada por el usuario.
- estado: reporte_estado_enum con valores PENDIENTE, RESUELTO, DESESTIMADO.
- fecha: TIMESTAMP. Fecha en que se creó el reporte.
- fecha_resolucion: TIMESTAMP. Fecha en que fue resuelto o desestimado (null si pendiente).

Entidad: Accion

Esta entidad registra acciones relevantes del sistema con fines de auditoría.

Atributos principales:

- act_id: UUID. Identificador único. Llave primaria.
- usuario_id: UUID (FK, nullable). Usuario que realizó la acción (null si el usuario fue eliminado).
- accion: TEXT. Descripción del evento registrado.
- fecha: TIMESTAMP. Fecha y hora del registro.

Tablas de unión

- animal_vacuna: animal_id UUID (FK), vacuna_id UUID (FK). Relaciona animales con sus vacunas.
- animal_padecimiento: animal_id UUID (FK), padecimiento_id UUID (FK). Relaciona animales con sus padecimientos.

Relaciones

- Un Usuario (cuidador) puede publicar muchos Animales (1:N).
- Un Usuario (adoptante) puede manifestar interés en muchos Animales (N:M a través de Interés con llave compuesta).
- Un Animal puede tener muchas manifestaciones de interés (1:N).
- Un Animal pertenece a exactamente un Usuario (cuidador) (N:1).
- Un Animal puede tener muchas Fotos (1:N a través de foto_animal).
- Un Animal puede tener muchas Vacunas (N:M a través de animal_vacuna).
- Un Animal puede tener muchos Padecimientos (N:M a través de animal_padecimiento).
- Un Usuario puede generar muchos Reportes (1:N).
- Un Animal puede tener muchos Reportes (1:N).
- Un Usuario puede tener muchas Acciones registradas (1:N).
- Un Usuario tiene un Código Postal asociado (N:1 conCodigoPostal).
- Un Animal puede tener una Raza del catálogo asociada (N:1 con Raza, opcional).

Consideraciones de seguridad

Se utiliza BCrypt como algoritmo de cifrado unidireccional con salt automático para el almacenamiento de contraseñas, lo que garantiza que dos contraseñas iguales produzcan hashes distintos y que el valor almacenado no sea recuperable en texto plano a partir del hash almacenado.

10. Limitaciones actuales

- El sistema no cuenta con despliegue en un entorno de producción; su uso se limita a un entorno local.

- La foto de perfil se sube después del registro (no durante), ya que se requiere que solo usuarios registrados suban archivos.
- El almacenamiento de imágenes depende de Cloudinary; sin credenciales configuradas, la subida de fotos no funciona.
- El panel de administración permite gestionar reportes, pero no incluye funcionalidades adicionales de moderación (edición de publicaciones de terceros, suspensión de cuentas).
- Los correos del sistema dependen de las credenciales de Gmail SMTP configuradas en el archivo .env; sin ellas, las funcionalidades de correo (2FA, verificación, recuperación, notificaciones) no operan.
- El cálculo de distancia entre usuarios y animales depende de la disponibilidad de coordenadas en la tabla codigo_postal, obtenidas previamente de Nominatim.
- Los tokens de recuperación de contraseña no tienen expiración explícita en la implementación actual.